Relaxation-Informed Training of Neural Network Surrogate Models

Calvin Tsay

Department of Computing, Imperial College London,
South Kensington, SW7 2AZ, United Kingdom.

Contributing authors: c.tsay@imperial.ac.uk;

Abstract

ReLU neural networks trained as surrogate models can be embedded exactly in mixed-integer linear programs (MILPs), enabling global optimization over the learned function. The tractability of the resulting MILP depends on structural properties of the network, i.e., the number of binary variables in associated formulations and the tightness of the continuous LP relaxation. These properties are determined during training, yet standard training objectives (prediction loss with classical weight regularization) offer no mechanism to directly control them. This work studies training regularizers that directly target downstream MILP tractability. Specifically, we propose simple bound-based regularizers that penalize the big-M constants of MILP formulations and/or the number of unstable neurons. Moreover, we introduce an LP relaxation gap regularizer that explicitly penalizes the per-sample gap of the continuous relaxation at training points. We derive its associated gradient and provide an implementation from LP dual variables without custom automatic differentiation tools. We show that combining the above regularizers can approximate the full total derivative of the LP gap with respect to the network parameters, capturing both direct and indirect sensitivities. Experiments on non-convex benchmark functions and a two-stage stochastic programming problem with quantile neural network surrogates demonstrate that the proposed regularizers can reduce MILP solve times by up to four orders of magnitude relative to an unregularized baseline, while maintaining competitive surrogate model accuracy.

1 Introduction

Neural network surrogate models have become a popular tool in mathematical optimization, enabling complicated or unknown functions to be replaced by trained parametric approximations that can then be embedded in optimization formulations [1–3]. Feedforward neural networks with rectified linear unit (ReLU) activations are particularly attractive for this purpose: the piecewise-linear structure of the ReLU function (and thus the combined network) allows the trained model to be encoded exactly in a mixed-integer linear program (MILP), enabling branch-and-bound global optimization [4–6]. Machine learning applications include NN verification/certification [7–10], counterfactual explanations [11, 12], reinforcement learning [13, 14], and model compression [15, 16]. Optimization applications include optimizing over black-box objectives [17–19], constraint learning [20, 21], and stochastic programming [22–24]. Toolkits such as JANOS [25], OMLT [26], and PySCIPOpt-ML [27] have helped popularize this approach across a range of engineering domains, including process design, energy systems, and planning [28–31]. We refer the reader to Huchette et al. [5] for a comprehensive overview of the intersection between ReLU neural networks and MILP.

For a given trained network, the complexity and tractability of associated MILP formulations depends is linked to its structural properties. In the standard big-M formulation [32–34], each hidden neuron with unknown activation state is encoded by introducing a binary variable. The number of these variables effectively dictates the combinatorial search space of the branch-and-bound search. Equally important is the strength of the continuous LP relaxation, obtained by relaxing each binary variable to a continuous variable in $[0, 1]$. The LP provides a bound on the MILP optimum at every node of the branch-and-bound tree, and a loose relaxation forces the solver to explore more nodes before certifying global optimality. Both the number of binary variables and the LP relaxation gap remain important even in more sophisticated formulations [35, 36], and can be controlled by the variable bounds, which in turn depend on the network parameters θ (obtained during training). Tight bounds can reduce big-M constants, stabilize neurons, and strengthen the LP relaxation, effectively resulting in more manageable MILP problems [37–39]. However, computational approaches for tightening bounds often scale poorly with network size (e.g., requiring solving optimization problems for each neuron), and these strategies are applied *after training*, with no mechanism to guide the network towards tractable structures during model training.

Standard neural network training minimizes a prediction loss and may include classical weight regularization such as ℓ_1 or ℓ_2 penalties. While inclusion of these regularizers can improve downstream MILP tractability [6], neither term in the training objective directly accounts for the downstream application(s) of the resulting surrogate model. A model trained to high accuracy may have many unstable neurons or loose LP relaxation bounds, making subsequent MILP-based optimization intractable. This decoupling of training and optimization is an important, but largely unexplored, source of inefficiency in the surrogate modeling pipeline.

This paper proposes a family of regularization terms that can be added to the standard training loss to explicitly target the factors governing MILP tractability. The key observation is that the pre-activation bounds are often (sub)differentiable functions of the network parameters θ and can therefore be incorporated into regularizers for gradient-based training. We derive the form and gradient of each proposed regularizer, and we establish formal relationships between them and the full derivative of the LP relaxation gap with respect to θ. Furthermore, we show that the gradient of the LP relaxation itself can be computed efficiently using sensitivity of parametric linear programs [40, 41] and incorporated into regularizers.

The main contributions of this paper are as follows.

1. We derive two bound propagation-based regularizers: $\mathcal{R}_{\text{BW}}$ (bound-width) and $\mathcal{R}_{\text{SN}}$ (stable-neuron). We provide closed-form expressions for their subgradients via automatic differentiation through the bound propagation. The bound widths prescribe the big-M constants of the MILP, and their recursive structure through the network depth can be exploited for gradient computations at the cost of a single additional forward pass per training step.
2. We introduce the LP relaxation gap regularizer $\mathcal{R}_{\text{LP}}$, which directly penalizes the per-sample continuous relaxation of the MILP at each training point. We derive and express its gradient in terms of the LP dual variables and the standard backpropagation gradient. An exact implementation via a straight-through estimator avoids the need for custom differentiation tools.
3. We establish a gradient decomposition (Proposition 3) showing that the combined regularizer $\mathcal{R}_{\text{BW}} + \mathcal{R}_{\text{LP}}$ approximates the full total derivative of the LP gap with respect to θ, capturing both the direct sensitivity through the constraint right-hand sides and the indirect sensitivity through the big-M constants via IBP.
4. We evaluate all regularizers on benchmark surrogate functions and on large-scale stochastic programming tasks, measuring their effect on the number of unstable neurons, LP relaxation gap, MILP node count, and solve time across a range of network architectures and regularization strengths.

The remainder of the paper is organised as follows. We first contextualize our contribution in relation to existing work in Section 1.1. Section 2 then introduces neural network models, the training objective, the big-M MILP formulation, and bound propagation. Sections 3 and 4 derive the bound-based regularizers and the LP gap regularizer, respectively. Section 5 briefly analyzes the combined regularizer and its relation to the total derivative. Computational experiments are reported in Section 6, and conclusions are drawn in Section 7.

1.1 Relation to existing work

The idea of training for downstream tractability MILP has parallels in the adversarial machine learning literature, where networks trained for certified robustness have been shown to exhibit fewer unstable neurons and tighter bounded output domains than their unregularized counterparts [42–45]. Here, 'robustness' refers to provable immunity to adversarial perturbations within a norm ball around each input at inference [8] or training [9, 46] time. More generally, the motivation of training a model with its downstream optimization use in mind is shared with the *decision-focused learning* literature [47], e.g., 'smart predict-then-optimize' [48] and task-based learning [49].

While the regularizers developed in Sections 3–4 share some technical components with the verification literature (notably differentiable bounds and penalties that target neuron stability), our motivation, formulation, and context differ in several important respects detailed below. On the other hand, while we consider a similar application as the decision-focused learning literature, the proposed regularization strategies purely target downstream MILP tractability, rather than improving the quality of decisions.

Application domain. Certified training methods target *adversarial robustness* of classifiers, typically defined as classification accuracy under worst-case perturbation in an ϵ-ball around each test image. Note that this problem can often be solved without finding the worst-case perturbation (i.e., completely solving a MILP): it requires only a successful worst-case perturbation or safe bound [8, 36]. On the other hand, the proposed methods primarily target *surrogate models for optimization*. In this setting, the network approximates a continuous function over a known box domain $\mathcal{X}$, and the downstream task is to solve a MILP over/involving the trained surrogate [4]. In contrast to the verification literature, the relevant metrics therefore include optimization properties such as LP relaxation gap and MILP solve time.

Training for certification. In certified training, intermediate variable bounds are used to compute an output-level bound, rather than as targets themselves [43, 45]. Nevertheless, in MILP intermediate bounds directly influence the tightness of the overall formulation, and we therefore quantify and regularize with total bound width across all hidden neurons (an objective with no analog in the certified training literature). Furthermore, we propose regularizers directly targeting the relaxation gap, which is entirely specific to the MILP surrogate setting. Most similar to the present work methodologically are regularizers targeting neuron stability, which can accelerate verification of classification networks [42, 50]. The present work studies stability regularizers for the MILP setting, where the network is embedded as an objective or constraint in a downstream optimization problem rather than verified for a fixed property. Finally, we note the certified training literature typically studies multi-class classifiers with cross-entropy loss and specification-based margins. Our setting involves scalar regression surrogates trained with MSE loss, where there is no class margin to certify, and the relevant relaxation is the continuous relaxation of the surrogate MILP, not the convex outer adversarial polytope. This definition of model quality gives a fundamentally different perspective to the tradeoff between accuracy and tractability.

Decision-focused learning. Many decision-focused learning methods modify the training loss of the surrogate model completely [48, 51], e.g., minimizing task loss or regret rather than prediction error alone. These modifications aim to improve the quality of decisions produced in downstream applications. Most similar to the present work in motivation are regularizers targeting gradients of the learned surrogate model, with the purpose of improving *optimization performance* in downstream (gradient-based) optimization [52–54]. The present paper takes an orthogonal approach within this broader family. Here, our objective is not to improve the quality of the solution found (which is also an important problem), but to reduce the computational cost of finding it via MILP. In other words, decision-focused learning assumes the downstream problem is solvable and asks how to improve solution quality; this paper assumes a useful surrogate and asks how to make it tractable.

2 Background

2.1 Neural network notation

We consider feed-forward neural networks (NNs), which are directed acyclic graphs comprising nodes/neurons structured into L hidden layers. At each layer $l = 1, ..., L+1$, the NN contains nodes that receive the outputs of nodes in the preceding layer $(l-1)$ as inputs. Each node then computes a weighted sum of its inputs (known as the preactivation), and applies a nonlinear activation function to this computed term. While many options for activation function are available, we focus on the ReLU activation function

$$y = \max(0, w^T x + b),$$

which is amenable to mixed-integer linear programming (MILP) formulations, given its piecewise-linear form [5].

Mathematically, we denote a feedforward neural network model as $f_\theta : \mathbb{R}^{n_0} \to \mathbb{R}$ with L hidden layers, ReLU activations, and a linear output layer. Denote the weight matrix and bias vector at layer ℓ by $W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ and $b^{(\ell)} \in \mathbb{R}^{n_\ell}$, respectively, for $\ell = 1, \ldots, L+1$. The collective parameter vector for the model is expressed as $\theta = \{(W^{(\ell)}, b^{(\ell)})\}_{\ell=1}^{L+1}$. Finding the values for θ, e.g., to best fit a given dataset, is referred to as *training* the NN model.

For an input $x \in \mathbb{R}^{n_0}$, evaluation of the neural network $f_\theta(x)$ is referred to as a *forward pass*. In particular, the forward pass computes:

$$z_j^{(\ell)} = W_j^{(\ell)} \hat{x}^{(\ell)}[\ell - 1] + b_j^{(\ell)}, \qquad \ell = 1, \ldots, L+1, \quad j = 1, \ldots, n_\ell, \tag{1}$$

$$\hat{x}_j^{(\ell)} = \mathrm{ReLU}(z_j^{(\ell)}) = \max(0,\, z_j^{(\ell)}), \qquad \ell = 1, \ldots, L, \tag{2}$$

with $\hat{x}^{(0)} = x$ being the input and $f_\theta(x) = z^{(L+1)}$ being the scalar output (note the lack of nonlinear activation at the output layer). We consider the input domain as a box $\mathcal{X} = \{x : x^{\mathrm{lb}} \le x \le x^{\mathrm{ub}}\}$, noting that in general, other constraints on x could be added in later steps.

2.2 Training the neural network

The neural network parameters are trained using training data $\{(x_i, y_i)\}_{i=1}^N$, e.g., samples drawn from a function we wish to approximate $g : \mathcal{X} \to \mathbb{R}$. Generally, we find values of θ by minimizing empirical loss over the training data. For regression tasks, this is often taken using the mean squared error (MSE):

$$\mathcal{L}_{\mathrm{MSE}} = \frac{1}{N} \sum_{i=1}^{N} \big(f_\theta(x_i) - y_i\big)^2.$$

To avoid overfitting (or otherwise guide the training process), regularization terms can be appended to the loss function:

$$\mathcal{L}(\theta) \;=\; \mathcal{L}_{\mathrm{MSE}} \;+\; \lambda\, \mathcal{R}(\theta), \tag{3}$$

where $\mathcal{R}(\theta)$ is a regularization term and $\lambda > 0$ controls the trade-off between accuracy and regularization. We refer the reader to Goodfellow et al. [55] for a comprehensive overview of this training paradigm.

Training the neural network is typically performed using gradient-based optimization methods, requiring the computation of $\nabla_\theta \mathcal{L}$, e.g., using back-propagation. We observe that gradients are therefore required for both terms in (3), $\nabla_\theta \mathcal{L}_{\mathrm{MSE}}$ and $\nabla_\theta \mathcal{R}(\theta)$. This study precisely aims to introduce regularizers $\mathcal{R}(\theta)$ targeting the MILP tractability of the resulting NN surrogate. In Section 3, we explicitly derive the form and gradient of each regularizer $\mathcal{R}$ we consider.

2.3 Mixed-integer optimization formulation

In contrast to the training of NNs (where the parameters θ are decision variables), optimization over a NN surrogate seeks to compute extreme cases for an *already trained* model. In other words, the parameters θ are fixed, and we optimize over f_θ as a fixed function (or embed it within constraints in a larger problem). This step therefore requires formulating f_θ over $\mathcal{X}$ as optimization constraints.

In MILP formulations, each ReLU unit is commonly modelled with a binary variable $a_j^{(\ell)} \in \{0,1\}$ indicating whether neuron j at layer ℓ is active ($a_j^{(\ell)} = 1$) or inactive ($a_j^{(\ell)} = 0$). Let $L_j^{(\ell)} \le z_j^{(\ell)} \le U_j^{(\ell)}$ be valid pre-activation bounds. The big-M formulation of $\hat{x}_j^{(\ell)} = \mathrm{ReLU}(z_j^{(\ell)})$ is [32–34]:

$$\hat{x}_j^{(\ell)} \ge z_j^{(\ell)}, \tag{4a}$$

$$\hat{x}_j^{(\ell)} \ge 0, \tag{4b}$$

$$\hat{x}_j^{(\ell)} \le z_j^{(\ell)} - L_j^{(\ell)}\,(1 - a_j^{(\ell)}), \tag{4c}$$

$$\hat{x}_j^{(\ell)} \le U_j^{(\ell)}\, a_j^{(\ell)}, \tag{4d}$$

for each hidden neuron (ℓ, j), $\ell = 1, \ldots, L$. The tractability of this formulation hinges on the values used for the bounds $L_j^{(\ell)}$ and $U_j^{(\ell)}$, i.e., the big-M coefficients [38]. Notice that smaller values for these coefficients yield tighter constraints (4c)–(4d). Ideally, these bounds are taken to be as tight as possible such that they remain valid, with $z_j^{(\ell)} \in [L_j^{(\ell)}, U_j^{(\ell)}]$. We note that several interesting alternatives to the big-M formulation have been proposed [35, 36], but it remains popular given its simplicity.

Remark 1 (Strength of continuous relaxation) MILP is often solved using branch-and-bound algorithms, which leverage a cheaper, continuous relaxation to bound the objective at each node of the search tree. The solver then explores the domain over decision variables by 'branching' until the gap between the best feasible objective value found and the tightest relaxation found falls below a given tolerance. A tighter, or *stronger*, relaxation can reduce this search tree considerably. The continuous relaxation of (4) is obtained by relaxing $a_j^{(\ell)} \in \{0,1\}$ to $a_j^{(\ell)} \in [0,1]$, yielding a linear program (LP) whose optimal value bounds the MILP optimum. The *LP relaxation gap* is the difference between this LP bound and the MILP optimum. Since the LP relaxation of (4c)–(4d) tightens as $|L_j^{(\ell)}|$ and $U_j^{(\ell)}$ decrease, the choice of bounds is a primary determinant of relaxation strength, and therefore MILP solve efficiency. Figure 1 illustrates the predictions and continuous relaxations for several NN models.

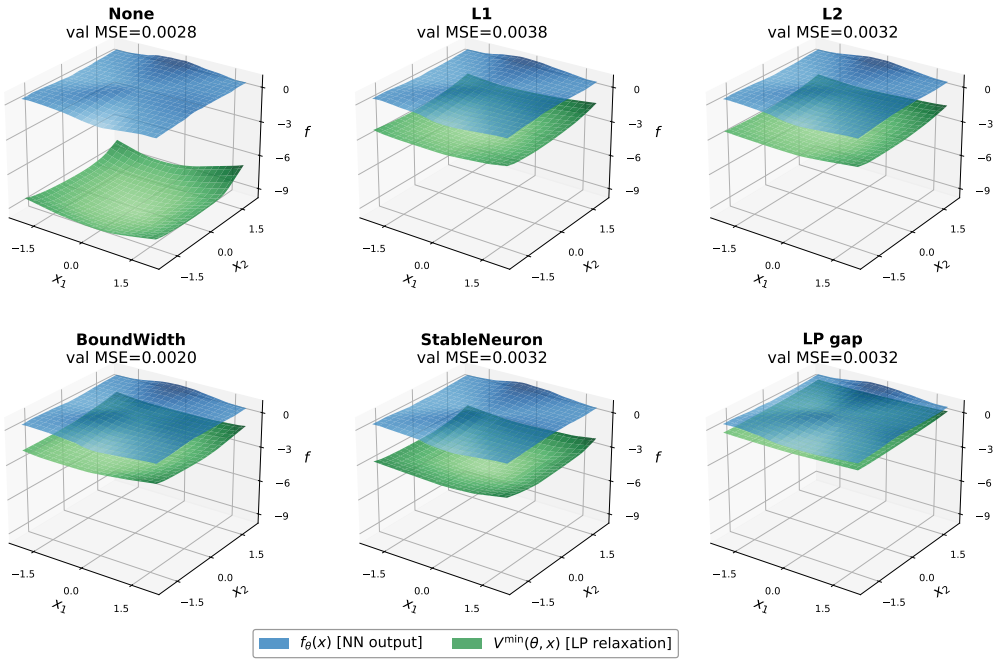

Fig. 1 Predictions and LP lower bounds for NN models with {32,32} hidden layers trained on the scaled Peaks function with two inputs (x_1,x_2), with output $f(x)$. Different regularizers are applied during training, with weights chosen to maintain validation MSE of similar scale.

Definition 1 (Neuron stability) A neuron is *stable active* if $L_j^{(\ell)} \geq 0$ (the ReLU never turns off), in which case $\hat{x}_j^{(\ell)} = z_j^{(\ell)}$ and $a_j^{(\ell)} = 1$ can be fixed. Likewise, a neuron is *stable inactive* if $U_j^{(\ell)} \leq 0$ ($\hat{x}_j^{(\ell)} = 0$, $a_j^{(\ell)} = 0$). A neuron is said to be *stable* if it is either stable active or inactive; for stable neurons, the value of $a_j^{(\ell)}$ is fixed, and no binary variable is required. The set of *unstable* neurons requiring a binary variable to formulate using (4) is therefore defined:

$$\mathcal{U} \;=\; \{(\ell, j) : L_j^{(\ell)} < 0 < U_j^{(\ell)}\}. \tag{5}$$

The number of (unfixed) binary variables in the MILP resulting from applying (4) to all neurons equals $|\mathcal{U}|$.

2.4 Obtaining and tightening bounds

Given the input domain $\mathcal{X} = [x^{\text{lb}}, x^{\text{ub}}]$, simple valid pre-activation bounds $L_j^{(\ell)}, U_j^{(\ell)}$ can be computed by applying interval arithmetic layer by layer. This is also referred to as interval bound propagation, or IBP. Let $\hat{l}^{(\ell)}, \hat{u}^{(\ell)}$ denote the post-activation (post-ReLU) bounds at layer ℓ, with the input layer defined by given bounds $\hat{l}^{(0)} = x^{\text{lb}}$, $\hat{u}^{(0)} = x^{\text{ub}}$.

Valid pre-activation bounds for a layer can be computed using interval arithmetic:

$$L_j^{(\ell)} = [W_j^{(\ell)}]^+ \hat{l}^{(\ell-1)} + [W_j^{(\ell)}]^- \hat{u}^{(\ell-1)} + b_j^{(\ell)}, \tag{6a}$$

$$U_j^{(\ell)} = [W_j^{(\ell)}]^+ \hat{u}^{(\ell-1)} + [W_j^{(\ell)}]^- \hat{l}^{(\ell-1)} + b_j^{(\ell)}, \tag{6b}$$

where the operators $[v]^+ = \max(v, 0)$ and $[v]^- = \min(v, 0)$ are applied element-wise. The ReLU function output is nonnegative, and the post-ReLU bounds for hidden layers can be further tightened:

$$\hat{l}_j^{(\ell)} = \max(L_j^{(\ell)}, 0), \qquad \hat{u}_j^{(\ell)} = \max(U_j^{(\ell)}, 0). \tag{7}$$

Interval arithmetic methods do not provide the tightest valid bounds in general, as dependencies between the input nodes are ignored. Propagating the resulting over-approximated bounds through the layers of a neural network leads to increasingly large over-approximations; in other words, propagating weak bounds through layers results in a model with significantly weaker continuous relaxation. Tighter bounds could potentially be obtained using optimization-based bound tightening (OBBT), i.e., solving an optimization problem with the objective set to minimize/maximize a particular pre-activation term to provide its bounds [37, 39]. To reduce the computational cost of OBBT problems, OBBT can be performed using relaxations or problem-based decompositions [38]. In contrast to interval arithmetic, bounds obtained using OBBT can incorporate variable dependencies. In this work, we focus on IBP bounds given their popularity.

A key observation is that the operations in the IBP recursion (6)–(7) are compositions of affine maps and element-wise $\max(\cdot, 0)$, similar to the ReLU forward pass. The IBP process is therefore subdifferentiable with respect to the NN parameters θ. The subgradients are well-defined almost everywhere and can be computed by automatic differentiation (e.g., in PyTorch or JAX). This property enables the IBP bounds to be incorporated directly into gradient-based training as differentiable regularization terms, as developed in the following sections.

3 MILP-informed regularization

We now introduce regularizers $\mathcal{R}(\theta)$ for use in the training objective (3) that target MILP tractability. For instance, Figure 1 shows the predictions and LP lower bounds for trained NN models on the Peaks function. We begin with standard shrinkage penalties, which serve as baselines, and then present two IBP-based regularizers that directly target the mechanisms governing MILP difficulty.

3.1 Shrinkage regularization

Shrinkage regularization is a strategy that aims to improve model generalizability and reduce overfitting by penalizing large parameter values, effectively 'shrinking' them towards zero. Shrinking the parameter values manages the bias-variance trade-off by introducing a small amount of bias to (significantly) reduce model variance. Common methods here include Ridge (L2) and Lasso (L1)

regression. These methods may produce models with tighter bounds, as shrinking the values of $W_j^{(\ell)}$ can directly improve the bounds obtained using (6). Plate et al. [6] find that increasing shrinkage regularization can produce neural networks with a lower number of linear regions, improving performance in downstream MILP. Manngård et al. [56] study methods using these regularizers to explicitly induce weight sparsity.

L1 regularization

$$\mathcal{R}_{\mathrm{L1}}(\theta) = \sum_{\ell=1}^{L+1} \big(\|W^{(\ell)}\|_1 + \|b^{(\ell)}\|_1 \big), \tag{8}$$

where $\|\cdot\|_1$ denotes the entry-wise ℓ_1 norm. This promotes weight sparsity and indirectly reduces IBP bound widths, as $U_j^{(\ell)} - L_j^{(\ell)}$ scales with $\|W_j^{(\ell)}\|_1$ (see (11) below). However, it does not account for the layered, recursive structure of bound propagation and treats all parameters uniformly regardless of their role in the MILP formulation.

L2 regularization

$$\mathcal{R}_{\mathrm{L2}}(\theta) = \sum_{\ell=1}^{L+1} \big(\|W^{(\ell)}\|_F^2 + \|b^{(\ell)}\|_2^2 \big), \tag{9}$$

where $\|\cdot\|_F$ denotes the Frobenius norm. Again, this can indirectly shrink big-M values but does not directly target bound widths or neuron stability.

3.2 Bound-width regularization

Define the *width* of the IBP pre-activation bound at neuron (ℓ, j) as $\Delta_j^{(\ell)} = U_j^{(\ell)} - L_j^{(\ell)}$. We introduce a bound-width regularizer, which simply penalizes the mean bound width obtained across all hidden neurons:

$$\mathcal{R}_{\mathrm{BW}}(\theta) \;=\; \frac{1}{\sum n_\ell} \sum_{\ell=1}^{L} \sum_{j=1}^{n_\ell} \Delta_j^{(\ell)} \;=\; \frac{1}{\sum n_\ell} \sum_{\ell=1}^{L} \sum_{j=1}^{n_\ell} \big(U_j^{(\ell)} - L_j^{(\ell)} \big). \tag{10}$$

Subtracting (6a) from (6b) gives the bound width at layer ℓ as:

$$\Delta_j^{(\ell)} \;=\; \big([W_j^{(\ell)}]^+ - [W_j^{(\ell)}]^- \big) \big(\hat{u}^{(\ell-1)} - \hat{l}^{(\ell-1)} \big) \;=\; |W_j^{(\ell)}|\, \Delta_{\mathrm{post}}^{(\ell-1)}, \tag{11}$$

where $|W_j^{(\ell)}|$ denotes the element-wise absolute value of the j-th row, and $\Delta_{\mathrm{post}}^{(\ell-1)}$ is the vector of post-ReLU bound widths at layer $\ell-1$. The post-ReLU bound widths satisfy $\Delta_{\mathrm{post},j}^{(\ell)} \le \Delta_j^{(\ell)}$ by (7), so the layer-wise recursion (11) shows that bound widths compound multiplicatively through the network depth.

Gradient.

Since the post-ReLU bound widths $\Delta_{\mathrm{post}}^{(\ell-1)}$ depend in turn on earlier layers through the recursion (6)–(7), the total gradient $\partial \mathcal{R}_{\mathrm{BW}} / \partial \theta$ captures the full chain of IBP bound propagation through the NN. In our experiments we directly implement (11) in PyTorch, and its gradient is computed automatically by PyTorch's reverse-mode automatic differentiation. Note that this requires implementing the (subdifferentiable) 'IBP forward pass,' i.e., propagating bounds through the layers of the neural network using (6)–(7). The computational cost is one IBP forward pass per training step. Note that more advanced OBBT propagation schemes may be incorporated as regularizers following the differentiable optimization procedures in Section 4.

Interpretation.

Including $\mathcal{R}_{\mathrm{BW}}$ as a regularization term in (3) explicitly penalizes the magnitude of big-M constants in the downstream MILP formulation. For an unstable neuron, we observe that $|L_j^{(\ell)}| \le \Delta_j^{(\ell)}$

and $U_j^{(\ell)} \leq \Delta_j^{(\ell)}$. Reducing $\Delta_j^{(\ell)}$ therefore simultaneously shrinks both big-M values in (4c)–(4d), tightening the LP relaxation. When the bounds are both positive or negative, the neuron is stable, and no binary variable is required (Definition 1).

Remark 2 An alternative view of $\mathcal{R}_{\text{BW}}$ is the direct penalization of the magnitude of big-M constants, i. e., the product of weight magnitudes and input bound ranges. In this case, this is exactly represented by (11), as IBP bound widths (composed recursively through the layers) are precisely the big-M constants. Nevertheless, in more sophisticated MILP formulations without big-M constants, corresponding regularization terms can still be derived based on the width of the involved bounds.

3.3 Stability regularization

While the inclusion of $\mathcal{R}_{\text{BW}}$ can strengthen the continuous LP relaxation by tightening bounds involved, the *combinatorial* difficulty of the MILP is governed by the number of binary variables, and therefore unstable neurons $|\mathcal{U}|$ in (5). Both the relaxation tightness and the number of discrete combinations in a search tree impact the efficiency of branch-and-bound algorithms. As given in Definition 1, neuron (ℓ, j) is unstable when its pre-activation bounds straddle zero: $L_j^{(\ell)} < 0 < U_j^{(\ell)}$. A naive approach could directly penalize the number of unstable neurons $|\mathcal{U}|$.

Nevertheless, knowing bounds $L_j^{(\ell)}$ and $U_j^{(\ell)}$ also informs us how 'close' a neuron is to being stable, e.g., how close the bounds are to zero. Based on this idea, we introduce a regularization term that penalizes the mean "distance to stability" to encourage stable nodes during training:

$$\mathcal{R}_{\text{SN}}(\theta) \;=\; \frac{1}{\sum n_\ell} \sum_{\ell=1}^{L} \sum_{j=1}^{n_\ell} \min\big([-L_j^{(\ell)}]^+,\, [U_j^{(\ell)}]^+\big), \tag{12}$$

where $[v]^+ = \max(v, 0)$. For a stable neuron ($L_j^{(\ell)} \geq 0$ or $U_j^{(\ell)} \leq 0$), at least one of $[-L_j^{(\ell)}]^+$ or $[U_j^{(\ell)}]^+$ is zero, so the contribution to $\mathcal{R}_{\text{SN}}$ is zero. For an unstable neuron, $[-L_j^{(\ell)}]^+ = |L_j^{(\ell)}|$ and $[U_j^{(\ell)}]^+ = U_j^{(\ell)}$, and the contribution to $\mathcal{R}_{\text{SN}}$ is $\min(|L_j^{(\ell)}|, U_j^{(\ell)}) > 0$. In other words, the regularizer pushes either $L_j^{(\ell)}$ upward toward zero (making the neuron stably active) or $U_j^{(\ell)}$ downward toward zero (making it stably inactive), whichever requires the smaller change. We note that, even if this does not force the neuron to be stable, pusing one of the bounds closer to zero may still produce a tighter continuous relaxation (Figure 2).

Proposition 1 (Subgradient of $\mathcal{R}_{\text{SN}}$) *The subgradient of* (12) *with respect to* θ *is*

$$\frac{\partial \mathcal{R}_{\text{SN}}}{\partial \theta} \;=\; \sum_{(\ell,j)\in\mathcal{U}} \begin{cases} -\dfrac{\partial L_j^{(\ell)}}{\partial \theta}, & \text{if } |L_j^{(\ell)}| < U_j^{(\ell)}, \\[2ex] \dfrac{\partial U_j^{(\ell)}}{\partial \theta}, & \text{if } U_j^{(\ell)} < |L_j^{(\ell)}|, \end{cases} \tag{13}$$

where $\partial L_j^{(\ell)}/\partial\theta$ *and* $\partial U_j^{(\ell)}/\partial\theta$ *are obtained from automatic differentiation through the IBP recursion. At the non-differentiable point* $|L_j^{(\ell)}| = U_j^{(\ell)}$, *any convex combination of the two cases is a valid subgradient.*

In our implementation, we use the PyTorch `torch.minimum` function, which handles the subgradient at the tie point $|L_j^{(\ell)}| = U_j^{(\ell)}$ automatically.

A related line of work aims to produce networks that are not only robust, but also *easy to verify exactly* using MILP-based solvers. Xiao et al. [42] identify weight sparsity and ReLU stability as two network properties that reduce exact verification time. They employ ℓ_1 regularization and small-weight pruning to promote sparsity, and introduce an alternative regularizer (termed RS loss in [42]) targeting ReLU stability. We denote this as an alternative stability regularizer:

$$\mathcal{R}_{\text{SN2}} \;=\; \frac{1}{\sum n_\ell} \sum_{\ell=1}^{L} \sum_{j=1}^{n_\ell} -\tanh\big(1 + U_j^{(\ell)} \cdot L_j^{(\ell)}\big), \tag{14}$$

where $U_j^{(\ell)}$ and $L_j^{(\ell)}$ are again upper and lower bounds on the pre-activation of neuron (ℓ, j). When both bounds have the same sign (stable neuron), the product $U_j^{(\ell)} \cdot L_j^{(\ell)}$ is positive and the penalty is small; when the bounds straddle zero (unstable neuron), the product is negative and the penalty increases. The authors found that adding this regularizer to the adversarial training objective reduces unstable neuron counts and yields considerable speedups in MILP-based verification time.

The stability regularizer $\mathcal{R}_{\text{SN}}$ (12) is conceptually related to the RS Loss (14) of Xiao et al. [42]: both encourage neurons to be stably active or inactive. Nevertheless, the formulations differ practically. Our proposed regularizer $\mathcal{R}_{\text{SN}}$ in (12) uses $\min([-L]^+, [U]^+)$, to directly measures the distance to stability and has a piecewise-linear structure. On the other hand, the RS Loss in (14), denoted here as $\mathcal{R}_{\text{SN2}}$, uses a smooth surrogate for sign agreement, which does not account for distance to stability. Moreover, in our setting $\mathcal{R}_{\text{SN}}$ and $\mathcal{R}_{\text{SN2}}$ can be combined with other regularizers to target complementary aspects of MILP difficulty.

Interpretation.

The regularizers $\mathcal{R}_{\text{SN}}$ and $\mathcal{R}_{\text{BW}}$ target different mechanisms of MILP. The former $\mathcal{R}_{\text{BW}}$ shrinks all bound widths uniformly, improving the strength of the continuous LP relaxation. On the other hand, $\mathcal{R}_{\text{SN}}$ concentrates its effect on the boundary at zero, aiming to eliminate (fix) binary variables from the formulation entirely. A trained NN could have tight bounds (small $\Delta_j^{(\ell)}$) that still straddle zero on many neurons, or wide bounds that happen to be one-sided (i.e., stable neurons). The two regularizers address related and complementary aspects of MILP difficulty and can be combined.

4 Relaxation-informed regularization

The regularization methods introduced in Section 3 all may help improve the strength of MILP reformulations of a NN surrogate model, albeit indirectly. In other words, they are heuristics aimed at producing tighter bounds or reducing the number of binary variables. Figure 2 illustrates the relaxation-related properties targeted by each regularizer. In this section, we consider a direct measure of relaxation quality: the LP relaxation gap itself.

For a given input $x_i \in \mathcal{X}$, the true network output is $f_\theta(x_i)$, which is uniquely determined by the forward pass (1)–(2). This unique solution is exactly encoded (for fixed input x_i) by the MILP constraints (4) when integrality is enforced. The LP relaxation, however, admits different output values because the relaxed binary variables $a_j^{(\ell)} \in [0, 1]$ allow intermediate neurons to deviate from their true ReLU outputs.

4.1 Pointwise LP relaxation gap

As mentioned above, the continuous relaxation of (4) obtained by relaxing integral constraints yields an LP that effectively provides a bound on the MILP optimum. We now derive a regularizer using the LP relaxation gap, i.e., the difference between the LP bound and the MILP solution (which gives the true NN output f_θ). For a fixed NN input x_i, we denote the LP relaxation value:

$$\begin{aligned} V^{\min}(\theta, x_i) \;=\; \min_{z^{(\ell)},\, \hat{x}^{(\ell)},\, a^{(\ell)}} \quad & z^{(L+1)} \\ \text{s.t.} \quad & (1),\ (4), \\ & a_j^{(\ell)} \in [0, 1], \\ & \hat{x}^{(0)} = x_i. \end{aligned} \tag{15}$$

The *pointwise LP gap* in the minimization direction is:

$$\delta^{\min}(\theta, x_i) \;=\; f_\theta(x_i) - V^{\min}(\theta, x_i) \;\geq\; 0, \tag{16}$$

since the LP relaxation can only under-estimate the minimum, i.e., $V^{\min} \leq f_\theta(x_i)$. An analogous quantity $\delta^{\max}(\theta, x_i) = V^{\max}(\theta, x_i) - f_\theta(x_i) \geq 0$ measures the gap in the maximization direction.

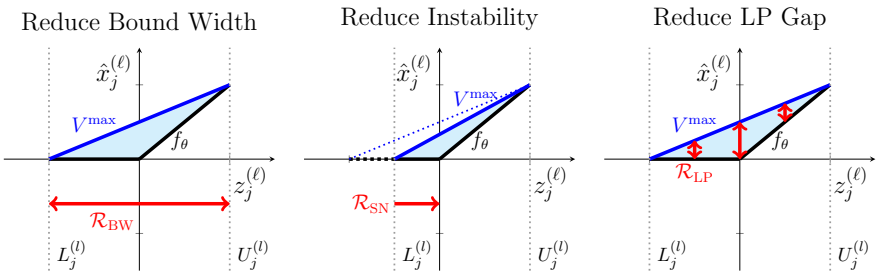

Fig. 2 Conceptual depiction of the various goals of MILP-related regularization.

The LP gap regularizer penalizes the average gap over a mini-batch B:

$$\mathcal{R}_{\text{LP}}(\theta) \;=\; \frac{1}{|B_s|} \sum_{i \in B_s} \delta(\theta, x_i), \tag{17}$$

where $B_s \subseteq B$ is an optional subsample of the training mini-batch to limit the number of LP solves per training step. In practice, we find that even $|B_s| = 1$ can achieve the desired effect. We use δ_i to denote $\delta_i^{\min}$, $\delta_i^{\max}$, or their sum (total LP gap), depending on the optimization context. For example, in surrogate-based problems where the NN output must be minimized, penalizing $\delta_i^{\min}$ is the natural choice, as it targets the gap relevant to the downstream MILP objective.

We note that in some settings surrogate models can have multiple output neurons, e.g., classification models or quantile neural networks [57, 58], complicating the definition of the LP relaxation value (15). For these models we propose quantifying the LP relaxation gap for a surrogate objective by projecting the vector of outputs $z^{(L+1)}$ onto a random vector, analogous to stochastic Sobolev training [54]. Following this approach, the objective function for (15) is replaced by $\omega^\top z^{(L+1)}$, where ω is a normalized randomly sampled vector. Averaging over many mini-batches would naturally encourage tightening in all possible output directions.

Interpretation

$\mathcal{R}_{\text{LP}}$ measures relaxation looseness at individual training points x_i, while the global LP gap (minimizing/maximizing over all $x \in \mathcal{X}$) measures the worst-case looseness over the domain $\mathcal{X}$. Including pointwise estimates at many training points is expected to generally tighten the relaxation over regions of the search space, e.g., sub-domains of a branch-and-bound search. Intuitively, the global relaxation may be tightened as well, though this is not guaranteed.

4.2 Differentiating through the LP solution

Computing the gradient $\partial \mathcal{R}_{\text{LP}} / \partial \theta$ requires differentiating the solution to the LP (15) with respect to the network parameters θ. The LP is a parametric linear program whose constraint data depend on θ. Writing this LP (15) in standard form:

$$\begin{aligned} V^{\min}(\theta, x_i) \;=\; \min_{y} \quad & c^\top y \\ \text{s.t.} \quad & A_{\text{eq}}(\theta)\, y = b_{\text{eq}}(\theta, x_i), \\ & G(\theta) y \le h(\theta), \\ & y^{\text{lb}} \le y \le y^{\text{ub}}, \end{aligned} \tag{18}$$

where y collects all primal variables $(z^{(\ell)}, \hat{x}^{(\ell)}, a^{(\ell)})$ across NN layers, c is the objective vector (in this case selecting the output neuron), the equality constraints encode the pre-activation definitions (1), and the inequality constraints encode the big-M constraints (4) with $a_j^{(\ell)} \in [0, 1]$.

At the LP solution, let $\nu^* \in \mathbb{R}^{m_{\text{eq}}}$ and $\mu^* \in \mathbb{R}^{m_{\text{ineq}}}$ denote the optimal dual variables for the equality and inequality constraints, respectively, with $\mu^* \ge 0$.

Proposition 2 (Sensitivity for parametric LP) *Suppose the LP (18) has a unique, non-degenerate optimal basis. Then the optimal value $V^{\min}$ is differentiable with respect to θ, and*

$$\frac{\partial V^{\min}}{\partial \theta} \;=\; (\nu^*)^\top \frac{\partial b_{\text{eq}}}{\partial \theta} \;-\; (\nu^*)^\top \frac{\partial A_{\text{eq}}}{\partial \theta}\, y^* \;+\; (\mu^*)^\top \frac{\partial h}{\partial \theta} \;-\; (\mu^*)^\top \frac{\partial G}{\partial \theta}\, y^*. \tag{19}$$

Proof To obtain these derivatives, we follow the approach of [59] and differentiate the KKT conditions. A similar analysis is also provided in Fiacco [41, Chapter 3.4]. In particular, the Lagrangian of (18) is given by:

$$L = c^\top y + \nu^\top \left(A_{\text{eq}}(\theta)\, y - b_{\text{eq}}(\theta, x_i)\right) + \mu^\top \left(G(\theta) y - h(\theta)\right). \tag{20}$$

The KKT conditions for stationarity, primal feasibility, and complementary slackness are:

$$\begin{aligned} c + A_{\text{eq}}(\theta)^\top \nu^* + G(\theta)^\top \mu^* &= 0, \\ A_{\text{eq}}(\theta)\, y^* - b_{\text{eq}}(\theta, x_i) &= 0, \\ D(\mu^*)\left(G(\theta) y^* - h(\theta)\right) &= 0, \end{aligned} \tag{21}$$

where the $D(\cdot)$ operator forms a diagonal matrix from a vector. To obtain a derivative, we assume (or approximate) the active-constraint set is locally constant, i.e., at a non-degenerate optimal basis, so the solution $[y^*(\theta), \nu^*(\theta), \mu^*(\theta)]$ is a smooth function of θ by the implicit function theorem applied to (21). We refer the reader to Fiacco [41, Chapter 2.4] for an overview of relevant implicit function theorem results. Since the objective vector c is fixed, we can first differentiate the objective $V^{\min} = c^\top y^*$, giving

$$\frac{\partial V^{\min}}{\partial \theta} \;=\; c^\top \frac{\partial y^*}{\partial \theta}. \tag{22}$$

We then substitute the stationarity condition from (21), giving

$$\frac{\partial V^{\min}}{\partial \theta} \;=\; (\nu^*)^\top A_{\text{eq}} \frac{\partial y^*}{\partial \theta} \;+\; (\mu^*)^\top G \frac{\partial y^*}{\partial \theta}. \tag{23}$$

Now, differentiating the primal feasibility conditions, $A_{\text{eq}}(\theta)\, y^*(\theta) = b_{\text{eq}}(\theta, x_i)$, gives:

$$A_{\text{eq}} \frac{\partial y^*}{\partial \theta} \;+\; \frac{\partial A_{\text{eq}}}{\partial \theta}\, y^* \;=\; \frac{\partial b_{\text{eq}}}{\partial \theta}. \tag{24}$$

For the inequality constraints, complementary slackness (21) gives $\mu_i^* > 0$ only when $G_i(\theta)\, y^* = h_i(\theta)$, i.e., the i-th constraint is active. Differentiating the active inequality constraints (noting that $\mu_i^* = 0$ for inactive constraints) therefore gives:

$$(\mu^*)^\top G \frac{\partial y^*}{\partial \theta} \;+\; (\mu^*)^\top \frac{\partial G}{\partial \theta}\, y^* \;=\; (\mu^*)^\top \frac{\partial h}{\partial \theta}. \tag{25}$$

Substituting (24) and (25) into (23) to eliminate $A_{eq} \cdot \partial y^*/\partial\theta$ and $G \cdot \partial y^*/\partial\theta$ yields (19). □

The result applies the familiar LP shadow-price interpretation (the rate of change of the optimum with respect to the right-hand side equals the dual variable) to perturbations in both the constraint matrix and the inequality right-hand side. In other words, the sensitivity of the optimal value to perturbations in constraint data can be computed from the optimal dual variables, without requiring differentiation through the `min` operator itself. The dual variables ν^* and μ^* are a standard output of LP solvers (e.g., as shadow prices from HiGHS or Gurobi). We refer the reader to [60, Chapter 5] for a more comprehensive treatment of parametric LPs and global LP sensitivity analysis.

While Proposition 2 gives a simple avenue to obtain sensitivities for simple, LP-based relaxations, more complicated formulations may also be used to produce bounds, e.g., convex NLP relaxations. Note that recent works [31, 61] study relaxations for nonlinear activation functions, another direction for future generalization beyond LP relaxations. The proposed regularizer may be generalized to these settings, e.g., following approaches to differentiate through nonlinear programs [59, 62, 63]. Moreover, traditional envelope theorems describe conditions for the value of a parameterized (nonlinear) optimization problem to be differentiable in the parameter and provide formulas for their derivatives [40, 41]. We note there is also a growing literature on software frameworks for differentiable optimization [64, 65].

Remark 3 (Envelope theorem versus KKT differentiation) An alternative route to $\partial V^{\min}/\partial\theta$ is to differentiate the KKT stationarity conditions implicitly. At the optimal basis, differentiating the stationarity conditions (21) with respect to θ yields a linear system involving the Jacobian $\partial y^*/\partial\theta$ of the optimal primal solution. Computing this Jacobian requires solving an $n_y \times |\theta|$ system at every training step, where n_y is the number of primal LP variables (scaling with network width and depth), and $|\theta|$ is the number of network parameters. The proposed formulation avoids this scaling entirely. Because $\partial\mathcal{L}/\partial y = 0$ at optimality (primal stationarity), the terms involving $\partial y^*/\partial\theta$ cancel in the total derivative, and the gradient collapses to the dual-weighted expression (19). In practice, we require only the dual variables ν^* and μ^*, which are already a standard solver output, (with no additional linear system to solve).

4.3 Application to the big-M LP

We observe that the LP constraints depend on the value of θ through two channels, as written in (18):

1. **Equality constraints (direct):** The pre-activation definitions $z^{(\ell)} = W^{(\ell)}\hat{x}^{(\ell)}[\ell-1] + b^{(\ell)}$ contribute $b^{(\ell)}$ to the right-hand side b_{eq} and $W^{(\ell)}$ to the constraint matrix A_{eq}.
2. **Inequality constraints (indirect):** The big-M values $L_j^{(\ell)}, U_j^{(\ell)}$ appearing in (4c)–(4d) contribute to both G and h, and their values depend on θ indirectly. For example, the bounds may be computed through IBP recursion (6).

Noting the similarity of channel (2) to the bound widths discussed in Section 3, in our implementation, we treat the big-M values $L_j^{(\ell)}, U_j^{(\ell)}$ as constants when differentiating through the LP, retaining only channel (1). This simplification is further motivated in Section 5, where we explicitly show that the omitted big-M sensitivity can be recovered by the bound-width regularizer when the two are combined.

Following this simplification, $L_j^{(\ell)}, U_j^{(\ell)}$ are treated as constants during differentiation, and the inequality constraint data G and h are independent of θ. The sensitivity (19) therefore reduces to:

$$\left.\frac{\partial V^{\min}}{\partial\theta}\right|_{L,U} = (\nu^*)^\top \frac{\partial b_{\text{eq}}}{\partial\theta} - (\nu^*)^\top \frac{\partial A_{\text{eq}}}{\partial\theta}\, y^*. \tag{26}$$

The equality constraints can be grouped by layer for ease of notation. At layer ℓ, the constraint $z_j^{(\ell)} - W_j^{(\ell)}\hat{x}^{(\ell)}[\ell-1] = b_j^{(\ell)}$ has dual variable $\nu_j^{(\ell)}$. Differentiating and plugging into (26) gives:

$$\left.\frac{\partial V^{\min}}{\partial b_j^{(\ell)}}\right|_{L,U} = \nu_j^{(\ell)}, \tag{27}$$

$$\left.\frac{\partial V^{\min}}{\partial W_{j,k}^{(\ell)}}\right|_{L,U} = \nu_j^{(\ell)} \cdot \hat{x}_k^{(\ell-1)*}, \tag{28}$$

where $\hat{x}_k^{(\ell-1)*}$ is the LP primal value of the post-activation variable at layer $\ell-1$ (for $\ell = 1$, these correspond to elements of the fixed input component x_i).

The gradient of the per-sample LP gap (16) is therefore approximated as:

$$\frac{\partial \delta_i^{\min}}{\partial\theta} = \frac{\partial f_\theta(x_i)}{\partial\theta} - \left.\frac{\partial V^{\min}}{\partial\theta}\right|_{L,U}, \tag{29}$$

where the first term is the standard backpropagation gradient.

4.4 Proxy implementation

Rather than implementing a custom backward pass for optimization problems as in [59], we construct a differentiable proxy tensor that has a gradient matching (27)–(28):

$$P(\theta) \;=\; \sum_{\ell=1}^{L+1} \big(\nu^{(\ell)}\big)^{\top} \big(W^{(\ell)}\, \hat{x}^{(\ell-1)*} + b^{(\ell)}\big), \tag{30}$$

where $\nu^{(\ell)}$ and $\hat{x}^{(\ell-1)*}$ are treated as fixed constants (detached from the computation graph) obtained from the LP solution, while the network parameters $W^{(\ell)}$ and $b^{(\ell)}$ remain in the computation graph. Observe that, by construction, the derivatives $\partial P/\partial\theta$ reproduce (27)–(28) exactly. Nevertheless, the forward-pass value of $P(\theta)$ does not match the true LP value $V^{\min}$, which we would like to include in the regularizer (17). Therefore, we apply the idea of a 'straight-through estimator,' i.e., a proxy derivative that is used in the backward pass only [66, 67]:

$$\widetilde{V}^{\min}(\theta) \;=\; P(\theta) - \mathrm{sg}\big[P(\theta)\big] + V^{\min}, \tag{31}$$

where $\mathrm{sg}[\cdot]$ denotes the *stop-gradient operator*. Specifically, $\mathrm{sg}[u]$ returns the same numerical value as u, but is treated as a constant during differentiation, i.e., $\partial\, \mathrm{sg}[u]/\partial\theta \equiv 0$. In automatic differentiation frameworks this is implemented by detaching the tensor from the computation graph (e.g. `u.detach()` in PyTorch).

The two passes of (31) behave differently by design. In the *forward pass*, $P(\theta)$ and $\mathrm{sg}[P(\theta)]$ evaluate to the same scalar p, so $\widetilde{V}^{\min} = p - p + V^{\min} = V^{\min}$. In other words, the forward pass returns the desired LP optimal value from the solver. In the *backward pass*, the stop-gradient removes the second term and the solver output $V^{\min}$ is a constant (solving the relaxation using an LP solver is not included in the computation graph), giving $\partial\widetilde{V}^{\min}/\partial\theta = \partial P/\partial\theta - 0 + 0 = \partial P/\partial\theta$. In other words, the backward pass returns the desired gradient (27)–(28). This proxy implementation avoids having to implement a custom backward pass for the proposed regularizer, while preserving both the correct function value and the correct gradient.

Figure 3 illustrates the pointwise LP relaxation gap $\delta^{\min}(\theta, x_i)$ in (16) for NN models trained with the various regularizers on a simple benchmark function. We observe that the LP gap regularizer can produce surrogate models with much tighter pointwise relaxations over the function domain.

5 Combining regularization strategies

Sections 3 and 4 introduce several regularization strategies that target different aspects of MILP difficulty when used downstream as surrogate models (Figure 2). A summary of the various proposed regularizers is given in Table 1. Computational costs for the various regularizers are given in Table 2.

Table 1 Summary of MILP- and relaxation-informed regularization strategies.

Regularizer	What it targets	Gradient w.r.t. θ
$\mathcal{R}_{\mathrm{BW}}$	Bound widths (big-M values)	$dL/d\theta$, $dU/d\theta$ via autodiff
$\mathcal{R}_{\mathrm{SN}}$	Number of binary variables (combinatorial difficulty)	$dL/d\theta$ or $dU/d\theta$ via autodiff (for unstable neurons)
$\mathcal{R}_{\mathrm{LP}}$	LP relaxation gap (L, U treated as constants)	LP dual variables
$\mathcal{R}_{\mathrm{BW}} + \mathcal{R}_{\mathrm{LP}}$	Both big-M values and LP gap	Approximates full $dV^{\max}/d\theta$ (see Proposition 3)

Here $|B_s|$ is the number of LP samples per batch (a tunable parameter to manage training overhead). The IBP forward pass has the same cost as a standard network forward pass (one

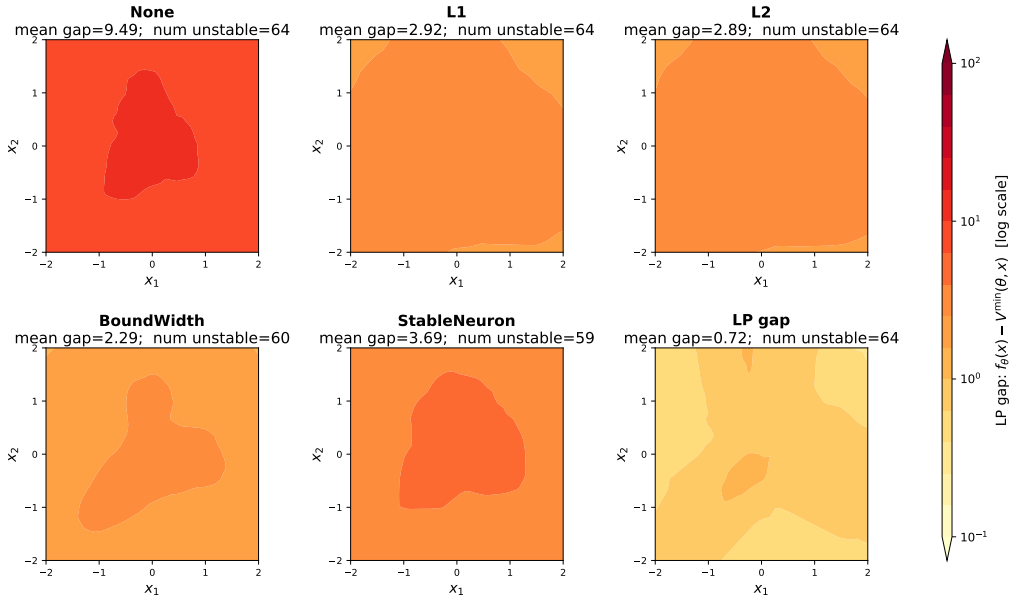

Fig. 3 LP gap for NN models with {32,32} hidden layers trained on the scaled Peaks function with two inputs (x_1,x_2), with output $f(x)$. Different regularizers are applied during training, with weights chosen to maintain validation MSE of similar scale.

Table 2 Per-step computational cost of each regularizer.

regularizer	Extra cost per training step	Gradient source
$\mathcal{R}_{\mathrm{L1}}$, $\mathcal{R}_{\mathrm{L2}}$	Negligible	Autograd
$\mathcal{R}_{\mathrm{BW}}$	1 IBP forward pass	Autograd through IBP
$\mathcal{R}_{\mathrm{SN}}$, $\mathcal{R}_{\mathrm{SN2}}$	1 IBP forward pass	Autograd through IBP
$\mathcal{R}_{\mathrm{LP}}$	$\lvert B_s\rvert$ LP solves	LP duals + autograd
$\mathcal{R}_{\mathrm{BW}} + \mathcal{R}_{\mathrm{LP}}$	1 IBP pass + $\lvert B_s\rvert$ LP solves	Both channels of (32)

matrix–vector product per layer). The LP solves are the dominant cost for $\mathcal{R}_{\mathrm{LP}}$; they can be parallelized across samples and potentially accelerated by warm-starting from the previous iterate.

5.1 Approximation of the total derivative

As observed in Section 4.3, the LP optimal value $V^{\max}$ depends on θ through two channels:

$$\frac{dV^{\max}}{d\theta} = \underbrace{\left.\frac{\partial V^{\max}}{\partial \theta}\right|_{L,U\ \text{const}}}_{\text{direct: constraint RHS}} + \underbrace{\sum_{(\ell,j)} \left(\frac{\partial V^{\max}}{\partial L_j^{(\ell)}} \frac{dL_j^{(\ell)}}{d\theta} + \frac{\partial V^{\max}}{\partial U_j^{(\ell)}} \frac{dU_j^{(\ell)}}{d\theta} \right)}_{\text{indirect: big-M values}}. \tag{32}$$

The first term is what $\mathcal{R}_{\mathrm{LP}}$ effectively computes via (26), as the bound widths (big-M values) are assumed constant. The second (omitted) term captures how changing θ alters the big-M constants $L_j^{(\ell)}, U_j^{(\ell)}$, which in turn affect the LP feasible region and hence the tightness of the LP relaxation. This second term factors as:

- $\partial V^{\max}/\partial L_j^{(\ell)}$ and $\partial V^{\max}/\partial U_j^{(\ell)}$: the sensitivity of the LP value to the big-M constants, given by the dual variables μ^* of the inequality constraints (4c)–(4d);
- $dL_j^{(\ell)}/d\theta$ and $dU_j^{(\ell)}/d\theta$: the gradients of the bound w.r.t. θ, e.g., obtained using IBP.

The bound-width regularizer $\mathcal{R}_{\text{BW}}$ penalizes $\sum_{(\ell,j)}(U_j^{(\ell)} - L_j^{(\ell)})$, whose gradient is:

$$\frac{\partial \mathcal{R}_{\text{BW}}}{\partial \theta} = \sum_{(\ell,j)} \left(\frac{dU_j^{(\ell)}}{d\theta} - \frac{dL_j^{(\ell)}}{d\theta} \right). \tag{33}$$

Comparing with the second (indirect) term in (32), we see that $\mathcal{R}_{\text{BW}}$ effectively provides a *surrogate* for the indirect big-M sensitivity path, with the LP dual multipliers $\partial V^{\max}/\partial L_j^{(\ell)}$ and $\partial V^{\max}/\partial U_j^{(\ell)}$ replaced by the uniform weights -1 and $+1$, respectively.

Proposition 3 (Combining $\mathcal{R}_{\text{BW}}$ and $\mathcal{R}_{\text{LP}}$ regularizers approximates the full gradient) *The combined regularizer $\mathcal{R}_{\text{LP}} + \alpha\,\mathcal{R}_{\text{BW}}$ produces the gradient:*

$$\frac{\partial \mathcal{R}_{\text{LP}}}{\partial \theta} + \alpha \frac{\partial \mathcal{R}_{\text{BW}}}{\partial \theta} = \underbrace{\left.\frac{\partial V^{\max}}{\partial \theta}\right|_{L,U\ const}}_{from\ \mathcal{R}_{\text{LP}}} + \alpha \underbrace{\sum_{(\ell,j)} \left(\frac{dU_j^{(\ell)}}{d\theta} - \frac{dL_j^{(\ell)}}{d\theta} \right)}_{from\ \mathcal{R}_{\text{BW}}}, \tag{34}$$

which approximates the total derivative (32) with the LP dual weights $\partial V^{\max}/\partial L_j^{(\ell)}$ and $\partial V^{\max}/\partial U_j^{(\ell)}$ replaced by α and $-\alpha$.

Remark 4 (Why not differentiate through the big-M values directly?) Computing the exact second term in (32) would require the LP dual variables μ^* for the inequality constraints as well as the full IBP Jacobian $dL/d\theta, dU/d\theta$. While feasible in principle, this doubles the information needed from each LP solve and couples the LP backward pass to the IBP backward pass. The combined $\mathcal{R}_{\text{LP}} + \alpha\,\mathcal{R}_{\text{BW}}$ avoids this coupling while still capturing both sensitivity paths, with α serving as a tunable proxy for the (unknown, sample-dependent) LP dual weights. The scalar α can be interpreted as a uniform "importance weight" for big-M tightness relative to constraint-RHS sensitivity.

6 Computational Results

To evaluate the regularization techniques proposed in Sections 3–4, we first consider the experimental settings of Plate et al. [6] and train NNs as surrogates for standard non-convex benchmark functions. We furthermore study quantile NNs as surrogates in stochastic programming applications, following Alcántara et al. [57]. We compare training and MILP performance on downstream optimization problems with different (combinations of) regularizers added during training.

6.1 Implementation

All experiments were run on a server equipped with AMD EPYC 7742 64-Core Processors. Each training and optimization run was allocated 8 CPU cores and 16 GB of memory. NN surrogate models and regularizers were implemented using PyTorch [68], and MILP optimization problems were solved using Gurobi v13.0.1 [69]. The LPs for the relaxation-based regularizer are implemented using `scipy.optimize` and solved using HiGHS [70]. The author acknowledges the use of Anthropic's Claude (v4.6 models) to assist with setting-up the server experimental environments. The content was reviewed by the author, who takes full responsibility for the final manuscript.

Although Gurobi can solve LPs, we use a HiGHS implementation for two reasons: first, it avoids the per-call overhead of constructing Gurobi model objects inside each training batch, which dominates runtime for the relaxed LPs encountered; and second, it keeps the entire training pipeline within open-source Python dependencies following the convention of machine learning software. The LP instances encountered during training consist of one LP per regularized sample, with the number of variables and constraints growing linearly in the total number of neurons. We found that HiGHS solves each these LP in milliseconds, but the cost accumulates over many mini-batches, which is reflected in the computational costs reported in Table 3. While our experiments are limited to CPU servers, an interesting direction for future work is to exploit GPU-based LP solvers [71, 72] during training, which could substantially reduce this overhead and integrate more naturally with GPU-based model training pipelines.

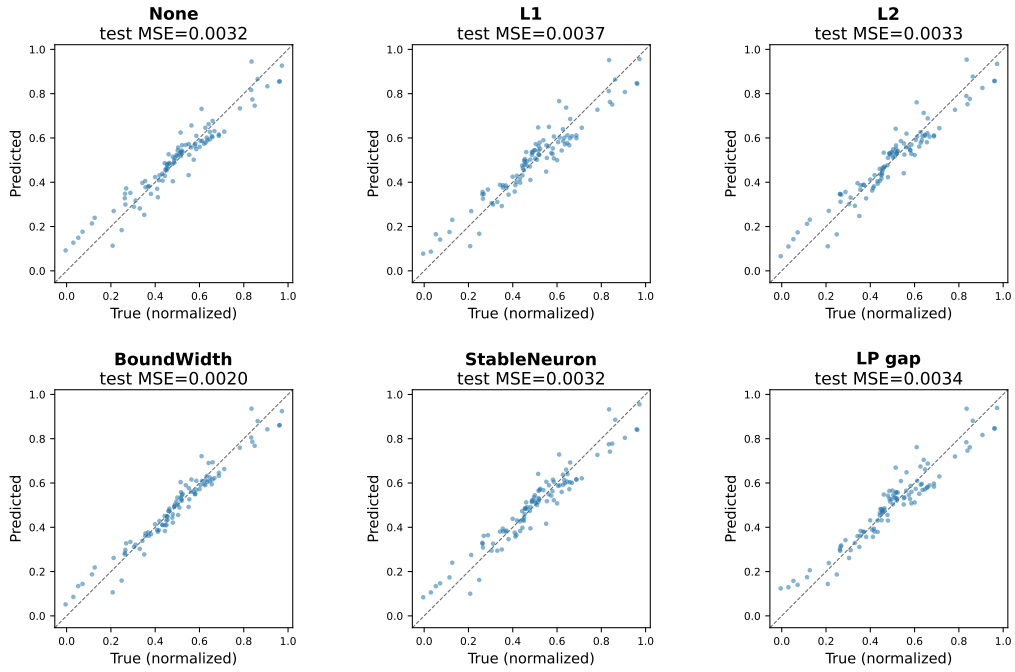

Fig. 4 Parity plots for NN models with {32,32} hidden layers trained on the scaled Peaks function with two inputs (x_1,x_2), with output $f(x)$. Different regularizers are applied during training, with weights chosen to maintain validation MSE of similar scale.

6.2 Direct Optimization over Surrogates

6.2.1 Benchmark Functions

We first study the direct minimization over a surrogate model output, i.e., solving the straightforward problem to minimize $f_\theta(x)$. We consider the benchmark functions and training settings used by Plate et al. [6] to facilitate comparison:

1. The Himmelblau function $f_{\text{himmelblau}} : [-5,5]^2 \to \mathbb{R}$, denoted as `himmelblau`, which has a global minimum of 0 at four points:

$$f_{\text{himmelblau}}(x) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2$$

2. The Peaks function $f_{\text{peaks}} : [-2,2]^2 \to \mathbb{R}$, denoted as `peaks`, which is multimodal with a unique global minimum of -6.551 [6]:

$$f_{\text{peaks}}(x) = -3(1-x_1)^2 \exp\left(-x_1^2 - (x_2+1)^2\right) - 10\left(\frac{x_1}{5} - x_1^3 - x_2^5\right)\exp(-x_1^2 - x_2^2) - \frac{1}{3}\exp\left(-(x_1+1)^2 - x_2^2\right)$$

3. The d-dimensional Ackley function $f_{\text{ackley}}, d : [-3.5, 3.5]^d \to \mathbb{R}$, denoted as `ackley-d`. The function is multimodal with a unique global minimum of 0:

$$f_{\text{ackley}}(x) = -20\exp\left(\frac{1}{5}\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2}\right) - \exp\left(\frac{1}{d}\sum_{i=1}^{d}\cos(2\pi x_i)\right) + \exp(1) + 20$$

As an illustrative example, we first train feedforward neural networks with two hidden layers of 32 neurons each with the various proposed regularizers, tuning the regularization penalties to ensure a similar validation MSE. The continuous relaxations are shown in Figure 1, the pointwise relaxation gaps in Figure 3, and prediction parity plots in Figure 4. This simple example allows

us to visually verify that the proposed regularizers may be tuned to improve relaxation tightness without significantly worsening prediction accuracy and/or generalization ability.

For the optimization studies below, we now partially follow the training setting of [6] and consider feedforward neural networks of with $\{2, 3, 5\}$ hidden layers of 25 neurons each. The models are trained on 100,000 samples for `peaks` and `himmelblau` and 150,000 samples for `ackley-2`, with all samples generated using Latin Hypercube sampling. We test some larger models including hidden layers of 50 neurons on the 5-dimensional Ackley function, `ackley-5`, where the number of samples is doubled to 300,000. Data are normalized, and 30% of the data are used as a test set to measure generalization ability. Networks are trained for 200 epochs using the Adam optimizer.

6.2.2 Training Results

Computational overhead.

Table 3 shows training-time ratios relative to the unregularized baseline. Shrinkage regularizers ($\mathcal{R}_{\text{L1}}, \mathcal{R}_{\text{L2}}$) add modest overhead (generally 1–2×), as they require only element-wise weight penalties. The bound-width and stability regularizers $\mathcal{R}_{\text{BW}}$, $\mathcal{R}_{\text{SN}}$ and $\mathcal{R}_{\text{SN2}}$ incur similar overheads to each other and slightly more than the shrinkage regularizers. We also observe their computational costs scale with network depth due to the layer-by-layer IBP propagation. The LP-based regularizer $\mathcal{R}_{\text{LP}}$ is the most expensive, at approximately 5–10×, reflecting the cost of solving one full LP relaxation of the network per regularization sample; the combined regularizer $\mathcal{R}_{\text{BW}} + \mathcal{R}_{\text{LP}}$ similarly reflects computational costs dominated by the LP component. Note these overheads are incurred only once during model training time and can potentially be amortized over usage in many downstream optimization instances.

Table 3 Training-time overhead of each regularizer relative to the unregularized baseline, averaged across `ackley-2`, `himmelblau`, `peaks`. Each cell shows $\bar{r} = \frac{1}{|\mathcal{B}|} \sum_{b \in \mathcal{B}} \frac{\bar{t}_{b,\text{reg}}}{\bar{t}_{b,\text{none}}}$, where $\bar{t}$ is the mean training time over seeds.

regularizer	λ	2-25-25-1	2-25-25-25-1	2-25-25-25-25-25-1
Baseline (t_0)	—	70.0 ± 12.3 s	83.9 ± 13.6 s	107.6 ± 22.2 s
None	—	1.00	1.00	1.00
$\mathcal{R}_{\text{L1}}$	10^{-4}	1.58 ± 0.31	1.62 ± 0.31	1.74 ± 0.30
	10^{-3}	1.19 ± 0.02	1.23 ± 0.02	1.25 ± 0.04
	10^{-2}	1.21 ± 0.04	1.23 ± 0.04	1.24 ± 0.04
$\mathcal{R}_{\text{L2}}$	10^{-4}	2.06 ± 0.73	1.55 ± 0.30	1.86 ± 0.65
	10^{-3}	1.28 ± 0.06	1.25 ± 0.06	1.30 ± 0.07
	10^{-2}	1.29 ± 0.12	1.23 ± 0.08	1.35 ± 0.07
$\mathcal{R}_{\text{BW}}$	10^{-4}	1.65 ± 0.26	1.77 ± 0.25	2.45 ± 0.40
	10^{-3}	1.42 ± 0.03	1.49 ± 0.06	1.70 ± 0.09
	10^{-2}	1.41 ± 0.04	1.52 ± 0.04	1.76 ± 0.05
$\mathcal{R}_{\text{SN}}$	10^{-4}	1.75 ± 0.20	2.08 ± 0.22	3.01 ± 0.77
	10^{-3}	1.59 ± 0.04	1.67 ± 0.07	1.91 ± 0.08
	10^{-2}	1.58 ± 0.01	1.64 ± 0.09	1.92 ± 0.10
$\mathcal{R}_{\text{SN2}}$	10^{-4}	2.38 ± 0.57	2.02 ± 0.41	2.19 ± 0.40
	10^{-3}	1.53 ± 0.03	1.57 ± 0.05	1.79 ± 0.08
	10^{-2}	1.51 ± 0.05	1.59 ± 0.03	1.83 ± 0.11
$\mathcal{R}_{\text{LP}}$	10^{-4}	6.06 ± 1.21	6.64 ± 0.64	10.92 ± 0.34
	10^{-3}	4.55 ± 0.09	6.38 ± 0.21	9.74 ± 0.43
	10^{-2}	4.59 ± 0.07	6.07 ± 0.23	9.09 ± 0.55
$\mathcal{R}_{\text{BW}} + \mathcal{R}_{\text{LP}}$	10^{-4}	5.92 ± 0.57	7.11 ± 1.26	10.28 ± 0.45
	10^{-3}	4.37 ± 0.03	5.14 ± 0.24	7.28 ± 0.38
	10^{-2}	4.01 ± 0.08	4.41 ± 0.14	5.43 ± 0.50

Table 4 Test MSE of each regularizer relative to the unregularized baseline, averaged across `ackley-2`, `himmelblau`, `peaks`. Cells show $\bar{r} = \frac{1}{|\mathcal{B}|}\sum_{b\in\mathcal{B}} \frac{\bar{m}_{b,\text{reg}}}{\bar{m}_{b,\text{none}}} \pm$ std, where $\bar{m}$ is the mean test MSE over seeds. Raw MSE values differ across benchmarks, so only the normalised ratio is shown. Values < 1 indicate lower test error than the baseline.

regularizer	λ	`2-25-25-1`	`2-25-25-25-1`	`2-25-25-25-25-25-1`
None	—	1.00	1.00	1.00
$\mathcal{R}_{\text{L1}}$	10^{-4}	4.51 ± 3.36	10.55 ± 7.11	32.86 ± 13.38
	10^{-3}	45.48 ± 43.78	106.86 ± 94.31	352.35 ± 235.47
	10^{-2}	148.46 ± 147.78	415.74 ± 416.71	1493.12 ± 1290.10
$\mathcal{R}_{\text{L2}}$	10^{-4}	1.92 ± 1.16	3.71 ± 1.52	12.09 ± 4.73
	10^{-3}	18.72 ± 19.17	35.29 ± 30.76	96.81 ± 58.84
	10^{-2}	84.11 ± 71.10	212.63 ± 170.74	760.82 ± 525.09
$\mathcal{R}_{\text{BW}}$	10^{-4}	0.54 ± 0.22	0.40 ± 0.13	0.78 ± 0.06
	10^{-3}	0.89 ± 0.38	0.99 ± 0.31	1.75 ± 0.23
	10^{-2}	3.15 ± 2.06	4.77 ± 2.06	11.04 ± 4.53
$\mathcal{R}_{\text{SN}}$	10^{-4}	0.74 ± 0.17	0.61 ± 0.21	0.80 ± 0.20
	10^{-3}	0.63 ± 0.26	0.68 ± 0.27	1.34 ± 0.10
	10^{-2}	2.28 ± 1.16	3.11 ± 1.04	6.47 ± 2.30
$\mathcal{R}_{\text{SN2}}$	10^{-4}	0.97 ± 0.02	0.97 ± 0.03	1.01 ± 0.02
	10^{-3}	0.93 ± 0.04	0.95 ± 0.03	1.00 ± 0.05
	10^{-2}	1.10 ± 0.12	0.99 ± 0.07	0.95 ± 0.02
$\mathcal{R}_{\text{LP}}$	10^{-4}	0.77 ± 0.35	0.65 ± 0.27	0.90 ± 0.14
	10^{-3}	0.98 ± 0.58	1.02 ± 0.47	2.17 ± 0.97
	10^{-2}	3.69 ± 3.18	7.20 ± 5.45	33.13 ± 28.23
$\mathcal{R}_{\text{BW}} + \mathcal{R}_{\text{LP}}$	10^{-4}	0.51 ± 0.25	0.47 ± 0.21	0.89 ± 0.16
	10^{-3}	1.45 ± 0.89	1.87 ± 0.83	3.75 ± 1.54
	10^{-2}	9.51 ± 9.58	12.47 ± 10.34	42.91 ± 28.30

Accuracy vs tractability tradeoff.

Table 4 shows normalized test MSE ratios relative to the unregularized baseline. Across all regularization methods, increasing the regularization penalty induces a reduced test accuracy as expected. Shrinkage regularizers ($\mathcal{R}_{\text{L1}}$, $\mathcal{R}_{\text{L2}}$) impose a steep accuracy penalty even at the moderate regularization strengths considered, with ratios exceeding $100\times$ at $\lambda = 10^{-3}$ for the deepest architectures. Note that, following convention, these shrinkage regularizers are not normalized by the number of model parameters, while the per-sample regularizers are averaged over number of samples.

On the other hand, the proposed bound-width ($\mathcal{R}_{\text{BW}}$) and stability ($\mathcal{R}_{\text{SN}}$) regularizers achieve ratios near, or even below unity at $\lambda = 10^{-4}$, indicating that mild regularization of the IBP bound widths may even provide a beneficial implicit regularization that simultaneously improves generalization and MILP tractability (though we do not claim this in general). The combined $\mathcal{R}_{\text{BW}} + \mathcal{R}_{\text{LP}}$ regularizer shows the same property at $\lambda = 10^{-4}$. Nevertheless, for these regularizers, accuracy again degrades as λ increases, most notably for the LP and combined regularizers. These results suggest that, in general regularizers provide a handle for tuning the tradeoff between surrogate model quality and the tractability of downstream optimization applications.

6.2.3 Optimization Results

Tables 5–7 report results for surrogate models trained with the various regularization strategies on two-dimensional benchmark functions. Four MILP tractability metrics are reported: number of unstable neurons $|\mathcal{U}|$, LP relaxation gap, MILP node count, and wall-clock MILP solve time. The unregularized baseline is computationally intractable on the deepest architectures for all benchmarks, consistently exceeding the 1800 s MILP time limit.

Overall, models trained with the proposed regularizers exhibit reduced MILP solve times, by up to four orders of magnitude relative to the baseline. Recall that, stronger regularization can further reduce MILP solve time at the expense of accuracy (Table 4). To show this performance tradeoff, rows are shaded in grey when the mean objective found in the downstream problem is

at least 5% higher than the objective found using the unregularized surrogate model. On the simple `himmelblau` function (Table 5), surrogate model training appears especially sensitive to the shrinkage regularizers, where regularization degrades downstream performance in all cases except the weakest L2 regularizer. Models trained with the bound-width regularizer weakly included considerably accelerate the downstream MILP solution time without affecting solution quality. For the slightly more complicated `peaks` function (Table 6), the shrinkage regularizers again generally degrade decision performance for smaller surrogate models. This trend is mitigated for the deepest model (five hidden layers), which is more over-parameterized. In this setting, we found that regularization with the combined bound-width and LP regularizer to greatly accelerate downstream MILP solution, including many settings where the MILP can be solved in a single node.

Overall, on these simple functions, $\mathcal{R}_{\text{BW}}$ at $\lambda = 10^{-3}$ reduces unstable neurons by roughly 50% on `himmelblau` and `peaks` (e.g., from 75 to 32–43 for the 3-layer architecture) and drives MILP times below 1 s across most architectures on the simpler benchmarks. The stability regularizer $\mathcal{R}_{\text{SN}}$ achieves similar reductions in $|\mathcal{U}|$ and MILP times, with slightly less aggressive compression of the LP gap as expected. The second stability regularizer $\mathcal{R}_{\text{SN2}}$ consistently produces worse models compared to $\mathcal{R}_{\text{SN}}$, suggesting the weaker gradient signal can make it less effective in practice. The combined $\mathcal{R}_{\text{BW}} + \mathcal{R}_{\text{LP}}$ regularizer is particularly effective at the lowest regularization strength ($\lambda = 10^{-4}$), where it simultaneously achieves the smallest $|\mathcal{U}|$ and near-zero LP gap, resulting in the best MILP solve times in most settings where solution quality is not degraded. For the challenging `ackley-2` function (Table 7), the combined regularizer again generally results in shortest downstream MILP solve times. Note that the shrinkage regularizers can also reduce MILP solve times in this setting without affecting solution quality, albeit to a lesser extent.

Interestingly, inclusion of the LP regularizer $\mathcal{R}_{\text{LP}}$ alone nearly eliminates the LP relaxation gap (often to < 0.01), but does not reduce the number of unstable neurons, since it only tightens the continuous relaxation without encouraging neuron stability. As a result, Gurobi must still branch on a large number of binary ReLU variables, and solve times remain significant on the harder instances. This complementarity motivates the combined regularizer: $\mathcal{R}_{\text{BW}}$ drives neurons toward stability (reducing $|\mathcal{U}|$), while $\mathcal{R}_{\text{LP}}$ tightens the LP gap, and together they compound to produce substantially smaller B&B trees (and MILP solve times). In summary, results on these two-dimensional benchmark functions show that, for simpler functions (where models are more overparameterized), downstream performance is more sensitive to regularization weights, especially shrinkage regularizers, and weak relaxation-informed regularization can greatly accelerate downstream performance. For the challenging Ackley function, (where models are less overparametrized), most regularization techniques accelerate downstream solution, with the powerful combined regularizer producing models with the best performance across most architectures.

Effect of function complexity and architecture depth.

The relative benefit of the proposed regularizers grows with both function complexity and network depth. On `himmelblau` (the simplest benchmark), $\mathcal{R}_{\text{BW}}$ at $\lambda = 10^{-3}$ is sufficient to reduce the 5-layer MILP from infeasible within the time limit to 0.23 s on average. On `ackley-2`, higher LP gaps in the baseline (e.g., 17.84 vs. 13.39 for `peaks` with two hidden layers) mean that reducing $|\mathcal{U}|$ alone is not sufficient at low λ, and the combined regularizer is needed for consistently fast solves.

Table 8 shows results for some larger NNs trained on the five-dimensional Ackley function. The L2 regularizer was omitted from these experiments, as we found it to be dominated by the L1 regularizer in previous experiments, similar to literature observations for this setting [6]. On `ackley-5` function, even with more neurons (up to 250 in the 50-wide architecture), the combined $\mathcal{R}_{\text{BW}} + \mathcal{R}_{\text{LP}}$ regularizer at $\lambda = 10^{-4}$ reduces the 5-layer solve time from > 1800 s to under 1 s while maintaining competitive surrogate accuracy. The standard L1 shrinkage regularizers offers less noticeable tractability improvements on these harder instances (larger NN models), though they are effective on the smaller models, e.g., three hidden layers (middle column of Table 8). This observation suggests that directly targeting the structure of the MILP embedding is essential for large gains in more challenging settings, where surrogate models are larger and less overparametrized.

Table 5 Results on the `himmelblau` benchmark across architectures and regularizers. Each regularizer family shows three rows for $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$. $|\mathcal{U}|$: mean number of unstable neurons; LP gap: mean LP relaxation gap; MILP nodes/time: mean branch-and-bound nodes and wall-clock time (s). **Bold** values indicate the best (lowest) result across all regularizers at each λ level per architecture; ties are all bolded. Shaded entries mark when the mean objective value found is worse (higher) than the unregularized baseline; such rows are excluded from best-value consideration.

regularizer	λ	2-25-25-1				2-25-25-25-1				2-25-25-25-25-25-1			
		$\lvert\mathcal{U}\rvert$	LP gap	MILP		$\lvert\mathcal{U}\rvert$	LP gap	MILP		$\lvert\mathcal{U}\rvert$	LP gap	MILP	
				nodes	time			nodes	time			nodes	time
None	—	50.0	23.93	18,530	4.08	75.0	53.91	841,316	243.87	125.0	350.81	$>$ 2,321,554	$>$ 1800
$\mathcal{R}_{\mathrm{L1}}$	10^{-4}	49.5	1.12	49	0.14	71.2	0.37	67	0.25	105.0	0.28	778	2.02
	10^{-3}	46.5	0.51	19	0.07	61.5	0.04	1	0.06	85.2	0.51	1,412	2.9
	10^{-2}	49.9	6.85	3,400	1.44	75.0	15.15	742,272	295.94	125.0	65.26	$>$ 2,088,555	$>$ 1800
$\mathcal{R}_{\mathrm{L2}}$	10^{-4}	39.0	0.29	32	0.18	53.8	0.07	148	0.39	**70.5**	0.01	205	**0.50**
	10^{-3}	43.5	0.21	1	0.04	65.0	0.13	416	0.48	94.6	1.14	112,459	73.40
	10^{-2}	49.4	6.12	5,610	1.64	73.5	10.84	207,382	70.27	123.8	72.41	2,443,711	1649.91
$\mathcal{R}_{\mathrm{BW}}$	10^{-4}	39.3	1.64	231	0.38	52.1	0.31	275	0.46	79.2	0.24	1,280	1.73
	10^{-3}	**23.6**	0.26	**1**	**0.03**	**32.7**	**0.13**	**14**	**0.07**	**51.2**	**0.02**	**41**	**0.23**
	10^{-2}	13.2	0.13	1	0.01	18.4	0.04	1	0.02	**28.8**	**0.01**	**1**	**0.03**
$\mathcal{R}_{\mathrm{SN}}$	10^{-4}	45.3	5.61	1,148	0.65	61.8	1.55	2,327	1.21	88.4	0.54	9,531	8.06
	10^{-3}	25.5	0.64	**1**	0.04	34.9	0.40	23	0.15	53.7	0.11	312	0.74
	10^{-2}	**13.4**	**0.35**	**1**	**0.01**	17.0	0.30	1	0.02	28.0	0.06	1	0.06
$\mathcal{R}_{\mathrm{SN2}}$	10^{-4}	50.0	24.15	15,001	3.32	74.8	52.72	1,028,957	232.30	124.0	334.55	$>$ 3,090,349	$>$ 1800
	10^{-3}	49.6	24.09	18,414	4.36	73.7	52.28	978,297	276.49	123.0	347.01	$>$ 2,574,826	$>$ 1800
	10^{-2}	48.4	23.70	18,444	5.11	72.5	51.46	822,372	284.34	122.5	329.29	$>$ 2,483,279	$>$ 1800
$\mathcal{R}_{\mathrm{LP}}$	10^{-4}	50.0	0.51	159	0.46	75.0	0.11	1,906	1.07	125.0	0.01	693,059	582.14
	10^{-3}	49.9	**0.02**	60	0.22	75.0	0.01	1,316	0.98	125.0	0.00	35,418	33.46
	10^{-2}	49.5	0.00	1	0.06	74.9	0.00	163	0.27	124.8	0.00	7,184	7.38
$\mathcal{R}_{\mathrm{BW}} + \mathcal{R}_{\mathrm{LP}}$	10^{-4}	**32.2**	**0.08**	**1**	**0.03**	**46.6**	**0.00**	**1**	**0.07**	77.2	**0.00**	**204**	0.60
	10^{-3}	18.9	0.00	1	0.01	28.6	0.00	1	0.02	46.4	0.00	1	0.05
	10^{-2}	11.8	0.00	1	0.01	14.1	0.00	1	0.01	20.8	0.00	1	0.01

Table 6 Results on the `peaks` benchmark across architectures and regularizers. Each regularizer family shows three rows for $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$. $|\mathcal{U}|$: mean number of unstable neurons; LP gap: mean LP relaxation gap; MILP nodes/time: mean branch-and-bound nodes and wall-clock time (s). **Bold** values indicate the best (lowest) result across all regularizers at each λ level per architecture; ties are all bolded. Shaded entries mark when the mean objective value found is worse (higher) than the unregularized baseline; such rows are excluded from best-value consideration.

regularizer	λ	2-25-25-1				2-25-25-25-1				2-25-25-25-25-25-1			
		$\|\mathcal{U}\|$	LP gap	MILP		$\|\mathcal{U}\|$	LP gap	MILP		$\|\mathcal{U}\|$	LP gap	MILP	
				nodes	time			nodes	time			nodes	time
None	—	50.0	13.39	10,547	2.37	74.9	32.07	462,620	117.38	124.9	232.06	$>$ 2,337,016	$>$ 1800
$\mathcal{R}_{\mathrm{L1}}$	10^{-4}	47.8	1.22	34	0.08	69.2	0.44	107	0.23	100.8	1.58	133,770	90.61
	10^{-3}	47.8	2.31	456	0.36	64.5	3.79	6,404	3.22	75.8	4.07	118,823	125.17
	10^{-2}	49.5	2.61	566	0.43	74.0	2.95	5,058	2.48	124.7	10.64	858,981	613.41
$\mathcal{R}_{\mathrm{L2}}$	10^{-4}	42.1	1.00	86	0.23	60.5	0.29	622	0.51	**79.0**	0.02	649	0.89
	10^{-3}	44.0	0.29	42	0.07	59.8	0.81	2,680	1.73	72.2	0.24	1,606	5.25
	10^{-2}	50.0	4.01	1,726	0.80	75.0	7.74	36,476	17.60	125.0	32.88	2,308,125	1341.62
$\mathcal{R}_{\mathrm{BW}}$	10^{-4}	44.0	2.07	396	0.49	58.5	0.83	671	0.72	91.6	0.39	990	2.11
	10^{-3}	29.8	0.82	**1**	0.04	43.1	0.41	**1**	0.08	64.7	0.10	30	0.17
	10^{-2}	**17.1**	0.50	**1**	**0.02**	23.1	0.27	**1**	0.02	37.8	0.02	**1**	**0.03**
$\mathcal{R}_{\mathrm{SN}}$	10^{-4}	45.4	4.03	799	0.60	62.4	1.86	1,999	1.46	92.7	0.88	2,406	4.11
	10^{-3}	34.1	1.00	**1**	0.06	44.0	0.61	**1**	0.10	67.4	0.27	450	0.57
	10^{-2}	19.6	0.83	**1**	0.02	24.3	0.42	**1**	0.03	37.8	0.16	**1**	0.06
$\mathcal{R}_{\mathrm{SN2}}$	10^{-4}	50.0	12.26	8,454	2.15	74.6	27.77	297,886	87.00	123.3	200.84	$>$ 3,083,439	$>$ 1800
	10^{-3}	46.7	10.59	5,319	1.43	68.7	25.24	171,206	52.98	115.0	179.48	3,168,132	1773.63
	10^{-2}	40.2	9.38	3,967	1.18	62.5	21.51	94,206	32.22	111.2	158.83	2,735,933	1653.35
$\mathcal{R}_{\mathrm{LP}}$	10^{-4}	50.0	0.24	**1**	0.05	74.9	0.01	568	0.39	124.9	**0.00**	51,325	70.36
	10^{-3}	50.0	0.00	**1**	0.05	75.0	0.00	11	0.13	124.8	**0.00**	107,905	123.91
	10^{-2}	50.0	**0.00**	**1**	0.04	74.9	0.00	**1**	0.09	125.0	0.00	40,729	47.21
$\mathcal{R}_{\mathrm{BW}} + \mathcal{R}_{\mathrm{LP}}$	10^{-4}	**38.9**	**0.12**	**1**	**0.05**	**55.6**	**0.00**	**1**	**0.10**	90.3	0.00	**189**	**0.60**
	10^{-3}	**26.3**	**0.00**	**1**	**0.01**	**41.1**	**0.00**	**1**	**0.03**	**59.4**	0.00	**1**	**0.12**
	10^{-2}	14.4	0.00	1	0.01	**21.6**	**0.00**	**1**	**0.01**	**29.3**	**0.00**	**1**	0.03

Table 7 Results on the `ackley-2` benchmark across architectures and regularizers. Each regularizer family shows three rows for $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$. $|\mathcal{U}|$: mean number of unstable neurons; LP gap: mean LP relaxation gap; MILP nodes/time: mean branch-and-bound nodes and wall-clock time (s). **Bold** values indicate the best (lowest) result across all regularizers at each λ level per architecture; ties are all bolded. Shaded entries mark when the mean objective value found is worse (higher) than the unregularized baseline; such rows are excluded from best-value consideration.

regularizer	λ	2-25-25-1				2-25-25-25-1				2-25-25-25-25-25-1			
		$\lvert\mathcal{U}\rvert$	LP gap	MILP		$\lvert\mathcal{U}\rvert$	LP gap	MILP		$\lvert\mathcal{U}\rvert$	LP gap	MILP	
				nodes	time			nodes	time			nodes	time
None	—	50.0	17.84	14,683	3.82	75.0	94.92	1,642,697	462.94	125.0	777.85	$>$ 2,558,663	$>$ 1800
$\mathcal{R}_{\mathrm{L1}}$	10^{-4}	50.0	6.15	2,963	1.11	74.8	17.13	474,665	115.66	123.3	88.96	1,752,605	1304.37
	10^{-3}	47.4	2.34	1,012	0.52	69.9	6.14	45,800	15.65	101.8	32.29	1,250,287	900.52
	10^{-2}	49.6	2.13	803	0.51	69.1	1.62	7,001	3.45	113.2	10.66	694,491	648.10
$\mathcal{R}_{\mathrm{L2}}$	10^{-4}	48.0	6.56	4,978	1.34	71.2	19.20	498,437	98.04	118.5	107.00	3,377,901	1721.21
	10^{-3}	46.5	3.03	1,223	0.61	71.8	8.88	78,488	21.36	117.0	48.85	1,934,706	967.87
	10^{-2}	48.7	2.54	1,072	0.82	73.3	5.50	37,752	15.01	124.7	40.25	2,450,682	1443.08
$\mathcal{R}_{\mathrm{BW}}$	10^{-4}	41.2	4.94	612	0.69	62.5	9.36	15,001	5.01	**99.3**	8.54	486,829	470.16
	10^{-3}	22.3	0.90	**1**	0.03	**38.8**	**2.03**	**102**	**0.25**	66.5	1.78	3,712	2.46
	10^{-2}	9.9	0.12	6	0.02	12.2	0.05	1	0.01	21.7	0.08	1	0.02
$\mathcal{R}_{\mathrm{SN}}$	10^{-4}	45.5	6.73	1,220	0.79	64.5	11.36	19,685	9.29	101.2	12.68	357,570	492.44
	10^{-3}	34.5	2.33	32	0.15	51.8	3.65	1,261	0.87	83.3	3.54	33,285	35.72
	10^{-2}	10.2	0.16	1	0.01	17.8	0.34	1	0.02	35.2	0.55	14	0.11
$\mathcal{R}_{\mathrm{SN2}}$	10^{-4}	49.7	16.86	16,744	4.14	74.9	87.16	1,170,921	271.89	124.5	740.97	$>$ 2,145,526	$>$ 1800
	10^{-3}	48.8	16.66	11,933	2.83	73.2	84.83	1,157,158	273.73	122.1	759.84	$>$ 2,670,470	$>$ 1800
	10^{-2}	45.9	16.72	12,411	3.04	70.5	84.66	1,078,148	218.33	120.3	707.98	$>$ 2,667,471	$>$ 1800
$\mathcal{R}_{\mathrm{LP}}$	10^{-4}	50.0	**0.81**	49	0.21	75.0	**0.11**	3,438	1.71	125.0	0.02	601,896	592.19
	10^{-3}	50.0	0.00	1	0.07	75.0	0.01	308	0.35	125.0	0.00	59,792	74.28
	10^{-2}	49.0	**0.00**	6	0.06	74.5	0.00	4	0.12	124.8	0.00	1,834	1.73
$\mathcal{R}_{\mathrm{BW}} + \mathcal{R}_{\mathrm{LP}}$	10^{-4}	**37.2**	0.88	**20**	**0.10**	**58.0**	0.21	**269**	**0.51**	99.8	**0.00**	**2,772**	**2.53**
	10^{-3}	**19.6**	**0.07**	5	**0.02**	33.2	0.00	1	0.02	**64.2**	**0.00**	**25**	**0.20**
	10^{-2}	**7.7**	0.01	**1**	**0.00**	**8.3**	**0.00**	**1**	**0.00**	**10.8**	**0.00**	**1**	**0.01**

Table 8 Results on the `ackley-5` benchmark across architectures and regularizers. Each regularizer family shows three rows for $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$. $|\mathcal{U}|$: mean number of unstable neurons; LP gap: mean LP relaxation gap; MILP nodes/time: mean branch-and-bound nodes and wall-clock time (s). **Bold** values indicate the best (lowest) result across all regularizers at each λ level per architecture; ties are all bolded. Shaded entries mark when the mean objective value found is worse (higher) than the unregularized baseline; such rows are excluded from best-value consideration.

regularizer	λ	5-25-25-25-25-25-1				5-50-50-50-1				5-50-50-50-50-50-1			
		$\lvert\mathcal{U}\rvert$	LP gap	MILP		$\lvert\mathcal{U}\rvert$	LP gap	MILP		$\lvert\mathcal{U}\rvert$	LP gap	MILP	
				nodes	time			nodes	time			nodes	time
None	—	125.0	376.81	2,851,019	1541.25	149.6	32.70	2,044,480	1363.88	249.9	1955.61	> 468,135	> 1800
$\mathcal{R}_{\mathrm{L1}}$	10^{-4}	84.8	2.06	166,609	180.14	122.8	0.97	152,175	204.79	204.2	32.71	241,319	1042.25
	10^{-3}	81.8	0.08	339	0.47	130.4	**1.55**	**474,476**	**611.46**	181.0	7.20	207,418	698.49
	10^{-2}	123.0	9.77	872,461	554.50	**148.3**	**5.13**	**566,103**	**788.14**	229.2	12.83	277,162	1153.38
$\mathcal{R}_{\mathrm{BW}}$	10^{-4}	67.8	0.56	523	0.95	**79.8**	**1.84**	**1,506**	**1.76**	94.2	1.10	2,089	3.81
	10^{-3}	34.6	0.08	1	0.02	43.2	0.32	1	0.05	49.6	0.13	1	**0.05**
	10^{-2}	16.3	0.00	0	0.00	16.8	0.00	1	0.00	22.6	0.00	0	0.01
$\mathcal{R}_{\mathrm{SN}}$	10^{-4}	73.9	1.03	1,379	1.62	86.8	1.99	1,950	2.30	107.7	1.93	34,844	25.64
	10^{-3}	**43.5**	0.14	**2**	**0.06**	54.4	0.60	46	0.20	64.8	0.18	103	0.35
	10^{-2}	18.6	0.02	1	0.01	21.1	0.01	1	0.01	29.1	0.01	1	0.02
$\mathcal{R}_{\mathrm{SN2}}$	10^{-4}	124.3	358.06	2,667,570	1439.55	134.4	24.52	1,201,306	728.03	246.8	1732.69	> 591,537	> 1800
	10^{-3}	123.2	361.40	2,899,113	1401.15	**117.6**	19.23	744,149	377.74	243.7	1809.90	> 503,628	> 1800
	10^{-2}	**122.1**	349.86	2,418,068	1253.44	119.8	21.80	356,002	221.30	240.5	1475.18	> 637,046	> 1800
$\mathcal{R}_{\mathrm{LP}}$	10^{-4}	125.0	0.00	13,136	8.33	149.9	0.00	13,686	9.17	250.0	**0.00**	10,110	20.54
	10^{-3}	124.9	**0.00**	96,528	41.03	149.9	0.00	13,889	6.00	249.8	0.00	200,495	231.59
	10^{-2}	124.9	**0.00**	**614,044**	**224.40**	149.8	0.00	48,391	30.20	249.8	**0.00**	**5,465**	**12.43**
$\mathcal{R}_{\mathrm{BW}} + \mathcal{R}_{\mathrm{LP}}$	10^{-4}	**66.2**	**0.01**	**5**	**0.14**	71.9	0.04	3	0.31	**87.6**	0.01	**155**	**0.52**
	10^{-3}	30.9	0.00	1	0.01	34.9	0.00	1	0.04	**45.0**	**0.00**	**1**	0.06
	10^{-2}	13.0	0.00	1	0.00	14.4	0.00	1	0.01	**31.0**	0.60	18,290	90.03

6.3 Optimization over Quantile Neural Networks

The previous section studied the proposed regularizers on single-output surrogate models, where the downstream MILP simply minimizes the predicted output. However, many practical applications involve more complicated settings, e.g., surrogate models can have multiple outputs and the downstream optimization problem can involve a more complex objective. To encompass some of these elements, we consider quantile neural networks (QNNs) applied to two-stage stochastic programming (2SP), an important setting in which QNN surrogates have been used to approximate the distribution of the second-stage 'recourse' function [57, 58]. QNNs are multi-output models that predict specific quantiles of a target distribution, enabling them to estimate uncertainty and produce prediction intervals instead of only point predictions.

We consider the capacitated facility location problem (CFLP), a classical 2SP in which binary first-stage decisions $y \in \{0,1\}^{n_f}$ determine which of n_f facilities to open, and the second-stage recourse allocates facility capacity to n_c customer demands that are revealed as random scenarios ξ. Patel et al. [24] study the problem using standard NN surrogates for the second-stage objective, and Alcántara et al. [57] use QNN surrogates to enable risk-aware, distributional modeling. Interestingly, Liu et al. [73] train input-convex NN surrogates to accelerate the downstream 2SP problem, which can then be formulated as an LP. Following the setup of Alcántara et al. [57], we train a QNN surrogate $f_\theta : \{0,1\}^{n_f} \to \mathbb{R}^K$ that, given a first-stage decision y, predicts $K = 50$ quantiles of the second-stage cost distribution at equally spaced intervals. The QNN is trained by minimizing the pinball (quantile regression) loss:

$$\mathcal{L}_{\text{pinball}}(\theta) = \frac{1}{NK} \sum_{i=1}^{N} \sum_{k=1}^{K} \max\big\{\tau_k(v_i - f_{\theta,k}(y_i)),\ (\tau_k - 1)(v_i - f_{\theta,k}(y_i))\big\}, \tag{35}$$

where v_i is the realized second-stage cost for the i-th training sample, generated by solving the recourse problem for a random demand scenario.

Once trained, the QNN surrogate replaces the expensive recourse computation in the first-stage optimization and enables distributional predictions such as Conditional Value-at-Risk (CVaR). The resulting 2SP is formulated as a MILP that minimizes a mean and/or CVaR-based objective over the predicted quantile outputs:

$$\min_{y \in \{0,1\}^{n_f}} \ c_f^\top y \ + \ (1 - \lambda_{\text{2SP}}) \frac{1}{K} \sum_{k=1}^{K} f_{\theta,k}(y) \ + \ \frac{\lambda_{\text{2SP}}}{|\mathcal{T}|} \sum_{k \in \mathcal{T}} f_{\theta,k}(y), \tag{36}$$

where c_f is the vector of first-stage costs, λ_{2SP} is the risk-aversion parameter, $\mathcal{T} = \{k : \tau_k \geq \alpha\}$ is the set of tail quantile indices, and α is the CVaR confidence level. This formulation includes both binary decision variables and the MILP encoding of the trained QNN, making tractability of the overall problem highly dependent on the structural properties of the surrogate model. We refer the reader to Alcántara et al. [57] for further details on the QNN-based 2SP framework.

6.3.1 Training Results

We study the `CFLP_50_50` instance ($n_f = n_c = 50$), following the instance generation procedure of Patel et al. [24], which is based in turn on Cornuéjols et al. [74]. We also consider an extended `CFLP_75_75` instance ($n_f = n_c = 75$), generated using the same procedure. The training dataset for each consists of 20,000 samples, with samples generated by drawing a random first-stage decision y and solving the second-stage recourse for a random demand scenario. For the `CFLP_50_50` and `CFLP_75_75` problems, 250 and 570 samples respectively reached the 600 s time limit, but feasible solutions were found for all of them. Data are split 80/20 into training and validation sets, normalized, and models are trained for 200 epochs with Adam.

We consider three architectures with $\{2, 3, 5\}$ hidden layers of 25 neurons each, denoted respectively as `X-25-25-50`, `X-25-25-25-50`, `X-25-25-25-25-25-50`. The input dimension equals n_f, and output dimensions is 50, corresponding to 50 quantiles. Each configuration is trained over 20 random seeds and tested in downstream MILPs as in (36). We omit the $\mathcal{R}_{\text{SN2}}$ regularizer for brevity, as it failed to produce tractable models in nearly all of the settings above, but reintroduce

Table 9 Pinball loss (mean ± std over seeds, scaled $\times 10^2$) on `cflp_50_50` for each regularizer and architecture. Values are computed on the held-out test set. **Bold** indicates the lowest pinball loss within each λ level per architecture, excluding shaded rows. Shaded entries exceed the unregularized baseline by more than 10%, indicating that the regularizer has degraded predictive quality.

regularizer	λ	`50-25-25-50` ($\times 10^2$)	`50-25-25-25-50` ($\times 10^2$)	`50-25-25-25-25-25-50` ($\times 10^2$)
None	—	3.03 ± 0.02	3.02 ± 0.03	3.06 ± 0.00
$\mathcal{R}_{\text{L1}}$	10^{-4}	$\mathbf{2.94 \pm 0.01}$	$\mathbf{2.89 \pm 0.01}$	$\mathbf{2.87 \pm 0.01}$
	10^{-3}	3.43 ± 0.14	3.26 ± 0.15	3.20 ± 0.15
	10^{-2}	30.77 ± 6.80	32.54 ± 7.27	34.60 ± 7.07
$\mathcal{R}_{\text{L2}}$	10^{-4}	2.99 ± 0.02	3.00 ± 0.02	2.97 ± 0.02
	10^{-3}	3.15 ± 0.02	3.06 ± 0.01	3.02 ± 0.01
	10^{-2}	22.82 ± 3.54	14.38 ± 9.06	4.77 ± 0.02
$\mathcal{R}_{\text{BW}}$	10^{-4}	3.01 ± 0.02	3.00 ± 0.03	2.98 ± 0.02
	10^{-3}	$\mathbf{2.97 \pm 0.02}$	$\mathbf{2.94 \pm 0.02}$	2.93 ± 0.02
	10^{-2}	$\mathbf{2.92 \pm 0.01}$	$\mathbf{2.88 \pm 0.01}$	2.86 ± 0.01
$\mathcal{R}_{\text{SN}}$	10^{-4}	3.02 ± 0.02	3.01 ± 0.03	2.99 ± 0.03
	10^{-3}	3.01 ± 0.02	2.99 ± 0.02	2.95 ± 0.02
	10^{-2}	2.93 ± 0.02	2.90 ± 0.02	$\mathbf{2.86 \pm 0.01}$
$\mathcal{R}_{\text{LP}}$	10^{-4}	3.04 ± 0.02	3.09 ± 0.03	3.07 ± 0.04
	10^{-3}	3.08 ± 0.03	3.11 ± 0.04	3.33 ± 0.11
	10^{-2}	3.08 ± 0.03	3.12 ± 0.05	8.76 ± 5.59
$\mathcal{R}_{\text{BW}} + \mathcal{R}_{\text{LP}}$	10^{-4}	3.03 ± 0.02	3.04 ± 0.02	3.00 ± 0.02
	10^{-3}	2.99 ± 0.02	2.98 ± 0.02	$\mathbf{2.91 \pm 0.03}$
	10^{-2}	2.92 ± 0.01	23.05 ± 8.71	23.18 ± 10.39

the L2 regularizer, as its performance in multi-output settings has not been studied. The multi-output LP relaxation gap regularizer $\mathcal{R}_{\text{LP}}$ uses random projections with non-negative directions, consistent with the non-negative weights in the downstream objective. The computational overheads of the proposed regularizers are independent of the specific training data; we observe costs largely similar to what was reported in Table 3 for the benchmark functions.

Tables 9–10 report the test pinball loss (scaled $\times 10^2$) for each regularizer and architecture combination. The unregularized baseline achieves a pinball loss of approximately 3.0 and 2.5 for the two problems respectively (scaled) across all architectures. The proposed regularizers $\mathcal{R}_{\text{BW}}$ and $\mathcal{R}_{\text{SN}}$ preserve or even slightly improve prediction quality at all regularization strengths considered; on both problems $\mathcal{R}_{\text{BW}}$ at $\lambda = 10^{-2}$ achieves the lowest pinball loss across architectures. In contrast, the shrinkage regularizers again suffer severe prediction degradation at higher regularization strengths, supporting our observation that these regularizers must be tuned and deployed more cautiously in practice. For example, $\mathcal{R}_{\text{L1}}$ at $\lambda = 10^{-2}$ inflates the pinball loss by an order of magnitude (to 30–35 on both problems), indicating that the network has collapsed to a near-constant function. $\mathcal{R}_{\text{L2}}$ exhibits similar degradation at $\lambda = 10^{-2}$. Any downstream MILP tractability "improvements" at these settings are therefore not attributable to the regularizer, but rather to the trivialization of the surrogate model.

In this setting a trained QNN surrogate is usable across downstream instances, e.g., Alcántara et al. [57] perform sensitivity studies across various levels of λ_{2SP} and α, while Ghilardi et al. [58] consider varying first-stage costs c_f. Given the lack of an obvious single objective, we consider the pinball loss (35) as a general metric of model decision quality across downstream optimization tasks: rows where the test pinball loss exceeds the baseline by more than 10% are shaded in grey in Tables 9–10. Moreover, they are excluded from best-value comparisons in Tables 11–12.

6.3.2 Optimization Results

Tables 11–12 report the same four MILP tractability metrics for the 2SP formulations (36): unstable neuron count $|\mathcal{U}|$, LP relaxation gap, branch-and-bound nodes, and wall-clock MILP solve time. We arbitrarily select $\lambda_{\text{2SP}} = 0.1$ for the risk-aversion parameter and $\alpha = 0.9$ is the CVaR confidence

Table 10 Pinball loss (mean ± std over seeds, scaled $\times 10^2$) on `cflp_75_75` for each regularizer and architecture. Values are computed on the held-out test set. **Bold** indicates the lowest pinball loss within each λ level per architecture, excluding shaded rows. Shaded entries exceed the unregularized baseline by more than 10%, indicating that the regularizer has degraded predictive quality.

regularizer	λ	75-25-25-50 ($\times 10^2$)	75-25-25-25-50 ($\times 10^2$)	75-25-25-25-25-25-50 ($\times 10^2$)
None	—	2.53 ± 0.04	2.49 ± 0.03	2.47 ± 0.02
$\mathcal{R}_{\text{L1}}$	10^{-4}	$\mathbf{2.40 \pm 0.01}$	$\mathbf{2.35 \pm 0.00}$	$\mathbf{2.33 \pm 0.01}$
	10^{-3}	2.88 ± 0.12	2.75 ± 0.18	2.58 ± 0.05
	10^{-2}	30.51 ± 6.23	30.81 ± 6.63	34.69 ± 6.43
$\mathcal{R}_{\text{L2}}$	10^{-4}	2.49 ± 0.02	2.48 ± 0.02	2.44 ± 0.02
	10^{-3}	2.60 ± 0.02	2.50 ± 0.01	2.46 ± 0.01
	10^{-2}	24.11 ± 0.11	22.10 ± 5.75	15.82 ± 9.60
$\mathcal{R}_{\text{BW}}$	10^{-4}	2.51 ± 0.03	2.46 ± 0.02	2.43 ± 0.02
	10^{-3}	$\mathbf{2.44 \pm 0.02}$	$\mathbf{2.40 \pm 0.01}$	2.37 ± 0.02
	10^{-2}	$\mathbf{2.37 \pm 0.01}$	$\mathbf{2.34 \pm 0.01}$	2.33 ± 0.01
$\mathcal{R}_{\text{SN}}$	10^{-4}	2.52 ± 0.03	2.48 ± 0.03	2.45 ± 0.03
	10^{-3}	2.50 ± 0.03	2.44 ± 0.03	2.40 ± 0.02
	10^{-2}	2.39 ± 0.02	2.34 ± 0.02	$\mathbf{2.31 \pm 0.01}$
$\mathcal{R}_{\text{LP}}$	10^{-4}	2.55 ± 0.03	2.56 ± 0.02	2.50 ± 0.03
	10^{-3}	2.61 ± 0.04	2.60 ± 0.04	2.79 ± 0.04
	10^{-2}	2.60 ± 0.06	2.65 ± 0.05	7.50 ± 4.08
$\mathcal{R}_{\text{BW}} + \mathcal{R}_{\text{LP}}$	10^{-4}	2.53 ± 0.02	2.50 ± 0.03	2.47 ± 0.03
	10^{-3}	2.47 ± 0.02	2.44 ± 0.02	$\mathbf{2.36 \pm 0.05}$
	10^{-2}	3.55 ± 5.23	26.95 ± 0.76	28.69 ± 0.75

level. The unregularized baseline again becomes increasingly intractable with network depth, with mean solve times in the `cflp_50_50` case study ranging from 2.3 s (2 hidden layers) to 702 s (5 hidden layers) and node counts growing from 5,235 to 270,161. These are magnified in the `cflp_75_75` case study to 4.6 s (2 hidden layers) and 1395 s (5 hidden layers), with node counts growing from 9,061 to 1,799,004, reflecting the more challenging response surface learned by the surrogate.

In these two settings, we again observe that the L1 regularizer can be effective at low weights ($\lambda = 10^{-4}$), but must be deployed cautiously as it can quickly degrade model prediction performance. In contrast, the bound-width regularizer $\mathcal{R}_{\text{BW}}$ improves downstream performance with less sensitivity to tuning. For example, at $\lambda = 10^{-2}$ $\mathcal{R}_{\text{BW}}$ reduces MILP solve times by 1–3 orders of magnitude across all architectures while maintaining the lowest pinball loss in both problem settings (Tables 9–10). On the deepest architectures, the MILP solution time drops several orders of magnitude, from 702 s and 1395 s, on the two problems respectively, to less than 1 s, with unstable neurons reduced from 125 to approximately 20. The stability regularizer $\mathcal{R}_{\text{SN}}$ at $\lambda = 10^{-2}$ achieves similar improvements without sensitivity to regularization weight tuning, reducing the 5-layer solve time to ≈0.35 s with 25–30 unstable neurons, though we again observe its LP gap reduction is slightly less aggressive.

We again observe the LP-based regularizer $\mathcal{R}_{\text{LP}}$ effectively reduces the LP gap but, as in the benchmark experiments, does not reduce the number of unstable neurons by itself. On the 2- and 3-layer architectures, this still yields meaningful acceleration of the downstream MILP (e.g., 0.56 s and 1.47 s vs. baselines of 2.3 s and 18 s in the `cflp_50_50` setting). However, on the deepest architectures $\mathcal{R}_{\text{LP}}$ alone is insufficient, matching our observations from the Ackley function above. Here, despite reducing the LP gap from 2.5×10^6 to 1.4×10^4 at $\lambda = 10^{-2}$ in the `cflp_50_50` setting), the network retains all 125 unstable neurons and still requires >100 s to solve. Notably, $\mathcal{R}_{\text{LP}}$ at $\lambda = 10^{-2}$ also degrades prediction quality on the 5-layer architecture (pinball loss 8.76 vs. baseline 3.06), perhaps reflecting the difficulty of the multi-output LP regularization at strong regularization strengths. The same trends can be seen in the larger problem. Alternatively, this could simply be an effect of the regularizer strength, as the shrinkage regularizers also degrade prediction quality at this weight.

The combined regularizer $\mathcal{R}_{\text{BW}} + \mathcal{R}_{\text{LP}}$ at $\lambda = 10^{-3}$ achieves a particularly effective balance of the above effects in both problem settings. For the 5-layer surrogate model architecture, it reduces $|\mathcal{U}|$ from 125 to >50 at a weight of $\lambda = 10^{-3}$, reduces the LP gap from $\mathcal{O}(10^6)$ to approximately 400 on both problems, and produces a MILP solvable in under 1 s without degrading (even improving) prediction quality. Across almost all architectures the combined regularizer consistently matches or improves upon the individual components, confirming the complementarity strengths of targeting both neuron stability (via $\mathcal{R}_{\text{BW}}$) and relaxation tightness (via $\mathcal{R}_{\text{LP}}$).

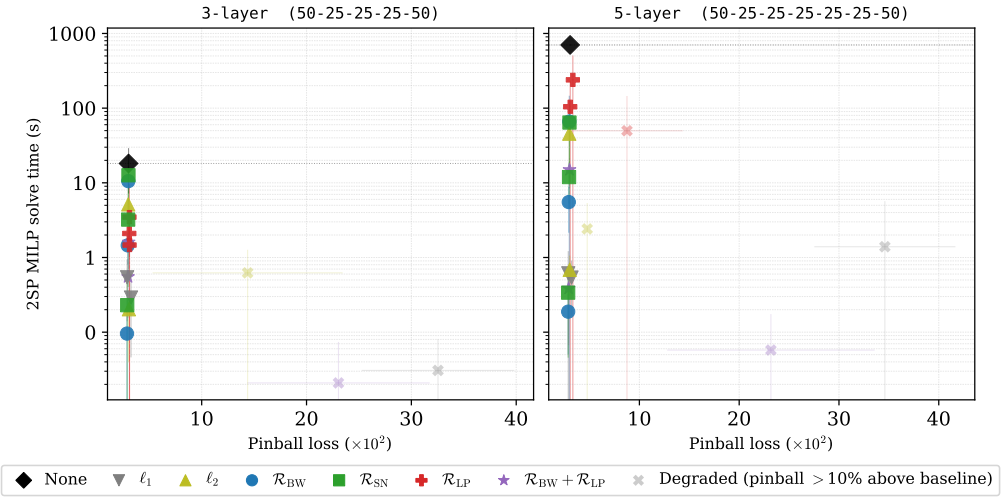

Fig. 5 Tradeoff between solution time of downstream MILP performance and pinball loss for QNNs in stochastic programming setting. Each plotted marker shows mean performance for a different regularizer and weight.

Comparison at equal prediction quality

The shading convention in Tables 9–10 and Tables 11–12 exposes a critical confound in naïve comparisons. As trends are largely consistent between the two problem settings, we focus our discussion on the `cflp_50_50` setting here. We observe that $\mathcal{R}_{\text{L1}}$ at $\lambda = 10^{-2}$ achieves the fastest MILP solve times in raw numbers (0.03 s), but this is only because the network has collapsed. In fact, Table 9 shows the pinball loss is 10× worse than the baseline. At comparable prediction quality, e.g., comparing $\mathcal{R}_{\text{L1}}$ at $\lambda = 10^{-4}$ (pinball 2.89, MILP time 0.54 s on the 3-layer model) against $\mathcal{R}_{\text{BW}}$ at $\lambda = 10^{-2}$ (pinball 2.88, MILP time 0.10 s), the proposed relaxation-informed regularizer achieves a 5× faster solve with 3× fewer unstable neurons for a QNN surrogate model the same predictive quality. This highlights the importance of accounting for prediction quality when evaluating MILP tractability improvements.

Figure 5 compares the pinball loss achieved by various regularizer configurations against the downstream MILP solve times in this setting, allowing us to visualize this tradeoff. Many points are clustered on the lefthand side of both plots (pinball loss close to the unnormalized baseline of ≈3). In this region of regularizers that maintain prediction quality, training with relaxation-aware regularizers $\mathcal{R}_{\text{BW}}$, $\mathcal{R}_{\text{SN}}$, and the combined regularizer sit at the bottom (lowest MILP solve times), forming the Pareto front. Improvements are most dramatic for the deeper QNN surrogate models. The shrinkage regularizers (and sometimes the combined regularizer) can produce model-collapse configurations where the regularizer dominates, resulting in a higher pinball loss values. These configurations are indicated using faded markers.

Limitations.

This work focuses on the big-M MILP formulation of ReLU networks; tailored regularizers for other formulations (e.g., ideal formulations [35] or partition-based formulations [36]) and other activation functions remain to be established. The bound-based regularizers $\mathcal{R}_{\text{BW}}$ and $\mathcal{R}_{\text{SN}}$ rely on IBP, which can produce increasingly loose bounds in deeper networks due to the recursive

over-approximation discussed in Section 2.4. The LP-based regularizer $\mathcal{R}_{\text{LP}}$ incurs a non-negligible training overhead (5–20×, Table 3), which scales with network size and limits its practicality for very large models, though this is less relevant for the surrogate model setting and cost is incurred only once during training. Furthermore, $\mathcal{R}_{\text{LP}}$ targets pointwise relaxation gaps, and global tightening is not guaranteed.

7 Conclusions

This paper introduced a family of relaxation-informed regularization strategies that target the downstream tractability of neural network surrogate models during training. Two bound propagation-based regularizers, $\mathcal{R}_{\text{BW}}$ (bound-width) and $\mathcal{R}_{\text{SN}}$ (stable-neuron), penalize the big-M constants and the number of unstable neurons, respectively, through automatic differentiation of the bounds computation. An LP relaxation gap regularizer $\mathcal{R}_{\text{LP}}$ directly targets the continuous relaxation tightness, with gradients derived from dual variables via the envelope theorem for parametric linear programs. Proposition 3 shows that combining $\mathcal{R}_{\text{BW}}$ and $\mathcal{R}_{\text{LP}}$ approximates the full total derivative of the LP gap with respect to the network parameters, capturing both the direct constraint sensitivity and the indirect big-M sensitivity.

Computational experiments on benchmark surrogate functions demonstrated that the proposed regularizers can reduce MILP solve times by up to four orders of magnitude while maintaining competitive modeling performance. On the two-stage stochastic programming case study with quantile neural networks, training with the bound-width regularizer $\mathcal{R}_{\text{BW}}$ could reduce the MILP solve time on the deepest architecture from over 700 s to under 1 s without degrading prediction quality, and the combined regularizer achieved similar acceleration, with complementary reductions in both unstable neuron count and LP gap. These results highlight an important practical consideration: classical shrinkage regularizers ($\mathcal{R}_{\text{L1}}$, $\mathcal{R}_{\text{L2}}$) can improve tractability at strong regularization weights, but this often requires tuning to avoid a collapse in predictive quality (rather than genuine improvement of the MILP formulation).

Several directions for future work are worth noting. The LP-based regularizer incurs a training overhead of approximately 5–20× due to repeated LP solves; exploiting GPU-based LP solvers [71, 72] could substantially reduce this cost and integrate with neural network training pipelines. Moreover, incorporating tighter bound propagation schemes (e.g., optimization-based bound tightening) as differentiable regularizers could yield further improvements, particularly for deeper architectures. Finally, extending the framework to other activation functions and to more complex downstream formulations beyond the big-M MILP remains an open direction.

Acknowledgements

Funding from the BASF/Royal Academy of Engineering Senior Research Fellowship is gratefully acknowledged.

Data Availability Statement

No new data were created for synthetic benchmarks. Data for stochastic programming applications were generated following `https://github.com/khalil-research/Neur2SP`.

Table 11 Results on the `cflp_50_50` QNN surrogate 2SP across architectures and regularizers. Each regularizer family shows three rows for $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$. $|\mathcal{U}|$: mean number of unstable neurons; LP gap: mean 2SP LP relaxation gap; MILP nodes/time: mean branch-and-bound nodes and wall-clock time (s) for the 2SP mean+CVaR MILP. **Bold** values indicate the best (lowest) result across all regularizers at each λ level per architecture; ties are all bolded. Shaded entries indicate regularizers whose test pinball loss exceeds the unregularized baseline by more than 10%, reflecting degraded predictive quality; such configurations are excluded from best-value consideration.

regularizer	λ	50-25-25-50				50-25-25-25-50				50-25-25-25-25-25-50			
		$\lvert\mathcal{U}\rvert$	LP gap	MILP		$\lvert\mathcal{U}\rvert$	LP gap	MILP		$\lvert\mathcal{U}\rvert$	LP gap	MILP	
				nodes	time			nodes	time			nodes	time
None	—	50.0	44005.06	5,235	2.31	75.0	196857.98	11,982	18.16	125.0	2469369.64	270,161	701.74
$\mathcal{R}_{\mathrm{L1}}$	10^{-4}	**32.2**	**319.71**	**1**	**0.39**	**55.0**	**593.19**	**4**	**0.54**	103.7	**7634.47**	**44**	**0.61**
	10^{-3}	32.0	110.63	12	0.29	**46.8**	**404.72**	145	0.29	75.7	829.68	52	0.54
	10^{-2}	32.4	37.42	1	0.03	37.0	102.94	1	0.03	50.0	222.49	384	1.39
$\mathcal{R}_{\mathrm{L2}}$	10^{-4}	46.1	11756.43	456	0.63	70.9	41684.35	7,079	5.14	117.5	237213.23	36,561	45.49
	10^{-3}	40.2	2326.55	**1**	**0.25**	54.3	6566.90	**1**	**0.20**	79.8	11557.11	56	0.68
	10^{-2}	44.5	744.48	1	0.06	60.9	1639.93	313	0.62	97.1	3470.08	4,049	2.41
$\mathcal{R}_{\mathrm{BW}}$	10^{-4}	48.1	26486.03	3,621	1.71	69.8	75976.64	9,747	10.53	108.2	217288.14	30,437	66.05
	10^{-3}	38.9	4776.56	193	0.49	57.4	17001.69	1,645	1.45	77.2	41766.83	5,948	5.52
	10^{-2}	16.4	207.46	**1**	**0.08**	**17.8**	**425.11**	**1**	**0.10**	**18.9**	**756.10**	4	**0.19**
$\mathcal{R}_{\mathrm{SN}}$	10^{-4}	46.9	27429.07	3,504	1.43	69.5	97827.09	9,981	12.62	107.6	411523.37	41,510	64.14
	10^{-3}	40.7	9861.86	1,458	1.14	59.0	30793.00	4,244	3.22	85.0	84640.76	7,524	11.94
	10^{-2}	21.3	935.90	**1**	0.17	25.4	1349.28	13	0.23	29.9	1201.37	**1**	0.34
$\mathcal{R}_{\mathrm{LP}}$	10^{-4}	49.8	4410.45	124	0.65	74.5	18490.36	2,325	2.10	125.0	22936.09	84,133	104.38
	10^{-3}	48.8	673.95	746	0.60	74.7	1171.38	5,972	3.47	125.0	49816.20	365,632	239.22
	10^{-2}	49.1	347.71	1,057	0.56	74.8	522.99	3,189	1.47	125.0	14303.72	64,183	49.79
$\mathcal{R}_{\mathrm{BW}} + \mathcal{R}_{\mathrm{LP}}$	10^{-4}	47.7	2390.24	26	0.58	68.6	10001.84	1,525	1.60	**101.6**	27172.28	8,130	14.94
	10^{-3}	**35.4**	**355.61**	311	0.52	48.8	429.38	146	0.55	**48.4**	**397.85**	**1**	**0.39**
	10^{-2}	**11.8**	**78.19**	**1**	0.15	8.2	83.49	1	0.02	22.9	182.73	1	0.06

Table 12 Results on the `cflp_75_75` QNN surrogate 2SP across architectures and regularizers. Each regularizer family shows three rows for $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}\}$. $|\mathcal{U}|$: mean number of unstable neurons; LP gap: mean 2SP LP relaxation gap; MILP nodes/time: mean branch-and-bound nodes and wall-clock time (s) for the 2SP mean+CVaR MILP. **Bold** values indicate the best (lowest) result across all regularizers at each λ level per architecture; ties are all bolded. Shaded entries indicate regularizers whose test pinball loss exceeds the unregularized baseline by more than 10%, reflecting degraded predictive quality; such configurations are excluded from best-value consideration.

regularizer	λ	75-25-25-50				75-25-25-25-50				75-25-25-25-25-25-50			
		$\|\mathcal{U}\|$	LP gap	MILP		$\|\mathcal{U}\|$	LP gap	MILP		$\|\mathcal{U}\|$	LP gap	MILP	
				nodes	time			nodes	time			nodes	time
None	—	50.0	76961.06	9,061	4.64	75.0	332696.67	13,755	20.16	125.0	3562051.76	1,799,004	1395.20
$\mathcal{R}_{\mathrm{L1}}$	10^{-4}	**32.2**	**1131.16**	**84**	**0.27**	**53.9**	**2322.02**	**121**	**0.48**	105.0	**12883.05**	**500**	**1.05**
	10^{-3}	34.2	193.47	1	0.34	47.9	1211.57	316	0.50	76.8	2946.37	396	0.81
	10^{-2}	31.2	0.00	1	0.03	38.2	41.94	1	0.03	27.2	0.00	1	0.02
$\mathcal{R}_{\mathrm{L2}}$	10^{-4}	48.4	26889.45	1,904	1.27	72.9	85918.82	9,295	8.19	120.0	495568.32	53,254	62.58
	10^{-3}	39.7	4500.89	**1**	**0.28**	55.4	10438.69	**1**	**0.24**	83.5	25823.26	967	1.34
	10^{-2}	44.9	1245.53	1	0.09	63.9	1895.88	147	0.24	92.8	2950.60	862	0.92
$\mathcal{R}_{\mathrm{BW}}$	10^{-4}	48.5	46063.34	8,071	4.28	71.0	140223.45	9,534	11.02	107.6	358786.38	71,025	91.14
	10^{-3}	40.8	10063.15	869	0.84	57.0	29386.24	3,820	2.06	69.5	59873.45	5,001	4.95
	10^{-2}	**15.6**	1342.98	**7**	**0.17**	**16.2**	1549.56	**1**	**0.20**	**20.9**	**1689.45**	**1**	**0.21**
$\mathcal{R}_{\mathrm{SN}}$	10^{-4}	48.1	54656.72	8,502	3.67	68.5	177112.92	9,151	11.40	107.0	649873.14	82,863	135.28
	10^{-3}	41.6	24314.73	3,984	2.18	57.7	55394.37	7,255	4.54	81.2	118008.83	6,244	16.25
	10^{-2}	20.1	1697.08	395	0.36	23.2	2063.29	45	0.38	26.9	2162.87	1	0.35
$\mathcal{R}_{\mathrm{LP}}$	10^{-4}	49.8	7710.15	1,626	1.20	74.9	34157.12	6,657	5.73	125.0	30153.51	146,887	134.11
	10^{-3}	48.6	1132.86	4,988	2.15	74.7	2150.41	50,828	16.63	125.0	27486.72	285,447	260.60
	10^{-2}	49.5	**363.43**	11,305	1.91	74.9	**1112.30**	29,829	8.05	125.0	23664.74	162,029	109.02
$\mathcal{R}_{\mathrm{BW}} + \mathcal{R}_{\mathrm{LP}}$	10^{-4}	47.6	4314.56	1,324	1.13	68.5	20778.39	3,851	2.63	**100.8**	37881.30	7,448	13.69
	10^{-3}	**35.9**	**556.51**	1,262	0.90	**50.1**	**720.99**	491	0.86	**43.0**	**432.37**	**8**	**0.33**
	10^{-2}	10.1	67.38	43	0.23	8.2	0.00	0	0.00	16.0	0.00	0	0.01

References

[1] Bertsimas, D., Margaritis, G.: Global optimization: a machine learning approach. Journal of Global Optimization **91**(1), 1–37 (2025)

[2] Bradley, W., Kim, J., Kilwein, Z., Blakely, L., Eydenberg, M., Jalvin, J., Laird, C., Boukouvala, F.: Perspectives on the integration between first-principles and data-driven modeling. Computers & Chemical Engineering **166**, 107898 (2022)

[3] Misener, R., Biegler, L.: Formulating data-driven surrogate models for process optimization. Computers & Chemical Engineering **179**, 108411 (2023)

[4] Grimstad, B., Andersson, H.: ReLU networks as surrogate models in mixed-integer linear programs. Computers & Chemical Engineering **131**, 106580 (2019)

[5] Huchette, J., Muñoz, G., Serra, T., Tsay, C.: When deep learning meets polyhedral theory: A survey. INFORMS Journal on Computing (2026)

[6] Plate, C., Hahn, M., Klimek, A., Ganzer, C., Sundmacher, K., Sager, S.: An analysis of optimization problems involving relu neural networks. Optimization and Engineering, 1–33 (2026)

[7] Botoeva, E., Kouvaros, P., Kronqvist, J., Lomuscio, A., Misener, R.: Efficient verification of relu-based neural networks via dependency analysis. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, pp. 3291–3299 (2020)

[8] Rössig, A., Petkovic, M.: Advances in verification of ReLU neural networks. Journal of Global Optimization **81**(1), 109–152 (2021)

[9] Sosnin, P., Müller, M.N., Baader, M., Tsay, C., Wicker, M.: Certified robustness to data poisoning in gradient-based training. arXiv preprint arXiv:2406.05670 (2024)

[10] Sosnin, P., Knapp, J., Kennedy, F., Collyer, J., Tsay, C.: Exact certification of data-poisoning attacks using mixed-integer programming. arXiv preprint arXiv:2602.16944 (2026)

[11] Kanamori, K., Takagi, T., Kobayashi, K., Ike, Y., Uemura, K., Arimura, H.: Ordered counterfactual explanation by mixed-integer linear optimization. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, pp. 11564–11574 (2021)

[12] Tsiourvas, A., Sun, W., Perakis, G.: Manifold-aligned counterfactual explanations for neural networks. In: International Conference on Artificial Intelligence and Statistics, pp. 3763–3771 (2024). PMLR

[13] Burtea, R., Tsay, C.: Constrained continuous-action reinforcement learning for supply chain inventory management. Computers & Chemical Engineering **181**, 108518 (2024)

[14] Ryu, M., Chow, Y., Anderson, R., Tjandraatmadja, C., Boutilier, C.: CAQL: Continuous action Q-learning. arXiv preprint arXiv:1909.12397 (2019)

[15] Benbaki, R., Chen, W., Meng, X., Hazimeh, H., Ponomareva, N., Zhao, Z., Mazumder, R.: Fast as chita: Neural network pruning with combinatorial optimization. In: International Conference on Machine Learning, pp. 2031–2049 (2023). PMLR

[16] Serra, T., Yu, X., Kumar, A., Ramalingam, S.: Scaling up exact neural network compression by ReLU stability. Advances in Neural Information Processing Systems **34**, 27081–27093 (2021)

[17] Perakis, G., Tsiourvas, A.: Optimizing objective functions from trained ReLU neural networks via sampling. arXiv preprint arXiv:2205.14189 (2022)

[18] Tong, J., Cai, J., Serra, T.: Optimization over trained neural networks: Taking a relaxing walk. In: International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research, pp. 221–233 (2024). Springer

[19] Tong, J., Zhu, Y., Serra, T., Burer, S.: Optimization over trained neural networks: Going large with gradient-based algorithms. arXiv preprint arXiv:2512.24295 (2025)

[20] Fajemisin, A.O., Maragno, D., Hertog, D.: Optimization with constraint learning: A framework and survey. European Journal of Operational Research **314**(1), 1–14 (2024)

[21] Maragno, D., Wiberg, H., Bertsimas, D., Birbil, Ş.İ., Hertog, D., Fajemisin, A.O.: Mixed-integer optimization with constraint learning. Operations Research **73**(2), 1011–1028 (2025)

[22] Dumouchelle, J., Julien, E., Kurtz, J., Khalil, E.B.: Neur2RO: Neural two-stage robust optimization. In: International Conference on Learning Representations (2023)

[23] Kronqvist, J., Li, B., Rolfes, J., Zhao, S.: Alternating mixed-integer programming and neural network training for approximating stochastic two-stage problems. In: International Conference on Machine Learning, Optimization, and Data Science, pp. 124–139 (2023). Springer

[24] Patel, R.M., Dumouchelle, J., Khalil, E., Bodur, M.: Neur2SP: Neural two-stage stochastic programming. Advances in neural information processing systems **35**, 23992–24005 (2022)

[25] Bergman, D., Huang, T., Brooks, P., Lodi, A., Raghunathan, A.U.: JANOS: an integrated predictive and prescriptive modeling framework. INFORMS Journal on Computing **34**(2), 807–816 (2022)

[26] Ceccon, F., Jalving, J., Haddad, J., Thebelt, A., Tsay, C., Laird, C.D., Misener, R.: OMLT: Optimization & machine learning toolkit. Journal of Machine Learning Research **23**(349), 1–8 (2022)

[27] Turner, M., Chmiela, A., Koch, T., Winkler, M.: PySCIPOpt-ML: Embedding trained machine learning models into mixed-integer programs. In: International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research, pp. 218–234 (2025). Springer

[28] Jalving, J., Ghouse, J., Cortes, N., Gao, X., Knueven, B., Agi, D., Martin, S., Chen, X., Guittet, D., Tumbalam-Gooty, R., *et al.*: Beyond price taker: Conceptual design and optimization of integrated energy systems using machine learning market surrogates. Applied Energy **351**, 121767 (2023)

[29] López-Flores, F.J., Ramírez-Márquez, C., Ponce-Ortega, J.M.: Process systems engineering tools for optimization of trained machine learning models: Comparative and perspective. Industrial & Engineering Chemistry Research **63**(32), 13966–13979 (2024)

[30] McDonald, T., Tsay, C., Schweidtmann, A.M., Yorke-Smith, N.: Mixed-integer optimisation of graph neural networks for computer-aided molecular design. Computers & Chemical Engineering **185**, 108660 (2024)

[31] Schweidtmann, A.M., Mitsos, A.: Deterministic global optimization with artificial neural networks embedded. Journal of Optimization Theory and Applications **180**(3), 925–948 (2019)

[32] Fischetti, M., Jo, J.: Deep neural networks and mixed integer linear optimization. Constraints **23**(3), 296–309 (2018)

[33] Lomuscio, A., Maganti, L.: An approach to reachability analysis for feed-forward relu neural

networks. arXiv preprint arXiv:1706.07351 (2017)

[34] Tjeng, V., Xiao, K.Y., Tedrake, R.: Evaluating robustness of neural networks with mixed integer programming. In: International Conference on Learning Representations (2017)

[35] Anderson, R., Huchette, J., Ma, W., Tjandraatmadja, C., Vielma, J.P.: Strong mixed-integer programming formulations for trained neural networks. Mathematical Programming **183**(1), 3–39 (2020)

[36] Tsay, C., Kronqvist, J., Thebelt, A., Misener, R.: Partition-based formulations for mixed-integer optimization of trained ReLU neural networks. Advances in neural information processing systems **34**, 3068–3080 (2021)

[37] Badilla, F., Goycoolea, M., Muñoz, G., Serra, T.: Computational tradeoffs of optimization-based bound tightening in relu networks. arXiv preprint arXiv:2312.16699 (2023)

[38] Sosnin, P., Tsay, C.: Scaling mixed-integer programming for certification of neural network controllers using bounds tightening. In: 2024 IEEE 63rd Conference on Decision and Control (CDC), pp. 1645–1650 (2024). IEEE

[39] Zhao, H., Hijazi, H., Jones, H., Moore, J., Tanneau, M., Van Hentenryck, P.: Bound tightening using rolling-horizon decomposition for neural network verification. In: International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research, pp. 289–303 (2024). Springer

[40] Milgrom, P., Segal, I.: Envelope theorems for arbitrary choice sets. Econometrica **70**(2), 583–601 (2002) https://doi.org/10.1111/1468-0262.00296

[41] Fiacco, A.V.: Introduction to Sensitivity and Stability Analysis in Nonlinear Programming. Academic Press, New York (1983)

[42] Xiao, K., Tjeng, V., Shafiullah, N.M., Madry, A.: Training for faster adversarial robustness verification via inducing ReLU stability. In: International Conference on Learning Representations (2019)

[43] Gowal, S., Dvijotham, K., Stanforth, R., Bunel, R., Qin, C., Uesato, J., Arandjelovic, R., Mann, T., Kohli, P.: On the effectiveness of interval bound propagation for training verifiably robust models. arXiv preprint arXiv:1810.12715 (2018)

[44] Mirman, M., Gehr, T., Vechev, M.: Differentiable abstract interpretation for provably robust neural networks. In: International Conference on Machine Learning, pp. 3578–3586 (2018). PMLR

[45] Zhang, H., Chen, H., Xiao, C., Gowal, S., Stanforth, R., Li, B., Boning, D., Hsieh, C.J.: Towards stable and efficient training of verifiably robust neural networks. In: International Conference on Learning Representations (2020)

[46] Sosnin, P., Wicker, M., Collyer, J., Tsay, C.: Abstract gradient training: A unified certification framework for data poisoning, unlearning, and differential privacy. arXiv preprint arXiv:2511.09400 (2025)

[47] Mandi, J., Kotary, J., Berden, S., Mulamba, M., Bucarey, V., Guns, T., Fioretto, F.: Decision-focused learning: Foundations, state of the art, benchmark and future opportunities. Journal of Artificial Intelligence Research **80**, 1623–1701 (2024)

[48] Elmachtoub, A.N., Grigas, P.: Smart "predict, then optimize". Management Science **68**(1), 9–26 (2022)

[49] Donti, P., Amos, B., Kolter, J.Z.: Task-based end-to-end model learning in stochastic optimization. Advances in neural information processing systems **30** (2017)

[50] Dvijotham, K., Gowal, S., Stanforth, R., Arandjelovic, R., O'Donoghue, B., Uesato, J., Kohli, P.: Training verified learners with learned verifiers. arXiv preprint arXiv:1805.10265 (2018)

[51] Tang, B., Khalil, E.B.: PyEPO: a PyTorch-based end-to-end predict-then-optimize library for linear and integer programming. Mathematical Programming Computation **16**(3), 297–335 (2024) https://doi.org/10.1007/s12532-024-00255-x

[52] Amos, B., Xu, L., Kolter, J.Z.: Input convex neural networks. In: International Conference on Machine Learning, pp. 146–155 (2017). PMLR

[53] Rosemberg, A.W., Garcia, J.D., Bent, R., Van Hentenryck, P.: Sobolev training of end-to-end optimization proxies. arXiv preprint arXiv:2505.11342 (2025)

[54] Tsay, C.: Sobolev trained neural network surrogate models for optimization. Computers & Chemical Engineering **153**, 107419 (2021)

[55] Goodfellow, I., Bengio, Y., Courville, A., Bengio, Y.: Deep Learning vol. 1. MIT press Cambridge, ??? (2016)

[56] Manngård, M., Kronqvist, J., Böling, J.M.: Structural learning in artificial neural networks using sparse optimization. Neurocomputing **272**, 660–667 (2018)

[57] Alcántara, A., Ruiz, C., Tsay, C.: A quantile neural network framework for two-stage stochastic optimization. Expert Systems with Applications **284**, 127876 (2025)

[58] Ghilardi, L.M., Patrón, G.D., Alcántara, A., Tsay, C.: Integrated design and scheduling of hydrogen processes under uncertainty: A quantile neural network approach. Industrial & Engineering Chemistry Research **64**(44), 21235–21250 (2025)

[59] Amos, B., Kolter, J.Z.: Optnet: Differentiable optimization as a layer in neural networks. In: International Conference on Machine Learning, pp. 136–145 (2017). PMLR

[60] Bertsimas, D., Tsitsiklis, J.N.: Introduction to Linear Optimization. Athena Scientific, Belmont, MA (1997)

[61] Wilhelm, M.E., Wang, C., Stuber, M.D.: Convex and concave envelopes of artificial neural network activation functions for deterministic global optimization. Journal of Global Optimization **85**(3), 569–594 (2023)

[62] Agrawal, A., Amos, B., Barratt, S., Boyd, S., Diamond, S., Kolter, J.Z.: Differentiable convex optimization layers. Advances in neural information processing systems **32** (2019)

[63] Pineda, L., Fan, T., Monge, M., Venkataraman, S., Sodhi, P., Chen, R.T., Ortiz, J., DeTone, D., Wang, A., Anderson, S., *et al.*: Theseus: A library for differentiable nonlinear optimization. Advances in Neural Information Processing Systems **35**, 3801–3818 (2022)

[64] Besançon, M., Dias Garcia, J., Legat, B., Sharma, A.: Flexible differentiable optimization via model transformations. INFORMS Journal on Computing **36**(2), 456–478 (2024)

[65] Rosemberg, A.W., Garcia, J.D., Pacaud, F., Parker, R.B., Legat, B., Sundar, K., Bent, R., Van Hentenryck, P.: A general and streamlined differentiable optimization framework. arXiv preprint arXiv:2510.25986 (2025)

[66] Bengio, Y., Léonard, N., Courville, A.: Estimating or propagating gradients through stochastic neurons for conditional computation. arXiv preprint arXiv:1308.3432 (2013)

[67] Yin, P., Lyu, J., Zhang, S., Osher, S., Qi, Y., Xin, J.: Understanding straight-through estimator in training activation quantized neural nets. In: International Conference on Learning Representations (2019)

[68] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al.: Pytorch: An imperative style, high-performance deep learning library. Advances in neural information processing systems **32** (2019)

[69] Gurobi Optimization, LLC: Gurobi Optimizer Reference Manual (2026). https://www.gurobi.com

[70] Huangfu, Q., Hall, J.J.: Parallelizing the dual revised simplex method. Mathematical Programming Computation **10**(1), 119–142 (2018)

[71] Applegate, D., Díaz, M., Hinder, O., Lu, H., Lubin, M., O'Donoghue, B., Schudy, W.: Practical large-scale linear programming using primal-dual hybrid gradient. Advances in Neural Information Processing Systems **34**, 20243–20257 (2021)

[72] Applegate, D., Hinder, O., Lu, H., Lubin, M.: Faster first-order primal-dual methods for linear programming using restarts and sharpness. Mathematical Programming **201**(1), 133–184 (2023)

[73] Liu, Y., Oliveira, F., Kronqvist, J.: ICNN-enhanced 2SP: Leveraging input convex neural networks for solving two-stage stochastic programming. arXiv preprint arXiv:2505.05261 (2025)

[74] Cornuéjols, G., Sridharan, R., Thizy, J.-M.: A comparison of heuristics and relaxations for the capacitated plant location problem. European Journal of Operational Research **50**(3), 280–297 (1991)